PANDUAN BELAJAR

Pahami projectmu dari awal sampai sidang

Prediksi beberapa label aroma dari struktur satu molekul

Baca pelan-pelan. Pahami satu tahap, lihat buktinya di data dan kode, lalu ceritakan lagi dengan bahasamu sendiri.

MASUK	PROSES	KELUAR
SMILES satu molekul	RDKit -> 2048 bit + 8 angka -> 109 classifier	109 skor -> ambang -> beberapa label aroma

Untuk hari pertama: cukup pengantar dan bab 01. Kamu belum perlu menjalankan Python.

Panduan ini memakai source aktif dan artefak tersimpan. Tidak ada training, tuning, pemasangan dependency atau perubahan pipeline saat bahan disusun.

Angka contoh diberi tanda ilustrasi. Angka penelitian ditautkan ke salinan hasil. Status aplikasi mengikuti audit 27 Oktober 2026.

Urutan isi

00. Mulai belajar dari sini . . . 3

1. Sebenarnya skripsi kita mau menjawab apa? . . . 5

2. Dari SMILES menjadi fitur dan target . . . 8

03. Dataset awal bentuknya seperti apa? . . . 11

04. Dari catatan sumber menjadi data bersih . . . 13

05. Mana yang dipakai belajar dan mana yang dipakai menguji? . . . 15

6 Model dan proses belajar . . . 17

7 Optuna validasi dan training final . . . 20

08 - Cara baca hasil: model kita bagusnya di bagian mana? . . . 22

9 Menjalankan project dan prediksi . . . 25

10 - Latihan jawab dosen pakai pemahaman sendiri . . . 28

11. Peta source dan cuplikan aktual . . . 32

12 Library dan cara membaca kode . . . 35

Cara baca: bab 00-10 berurutan. Bab 11-12 menjadi teman saat membongkar source. Simpan pertanyaan di CHECKLIST_BELAJAR.md.

00. Mulai belajar dari sini

Kamu nggak perlu langsung paham semua kode. Target awalnya cukup: kamu bisa cerita apa yang masuk, apa yang dikerjakan, dan apa yang keluar. Setelah alurnya nyambung, baru kita bongkar satu bagian setiap sesi.

Panduan ini diperiksa terhadap source, data, dan hasil yang tersedia pada **27 Oktober 2026**. Hasil utama yang dibahas adalah arsip eksperimen utama lima sumber, Morgan 2048 dan delapan deskriptor. Nama arsip V7 bukan nama metode. Eksperimen tambahan V8/V9 tidak dicampur ke tabel hasil utama.

Satu cerita yang perlu kamu pegang

Kamu memasukkan SMILES satu molekul. RDKit membaca strukturnya lalu mengubahnya menjadi angka. Untuk model terpilih, angkanya berupa **2048 bit Morgan + 8 deskriptor = 2056 fitur**. Angka ini masuk ke 109 classifier LightGBM, masing-masing untuk satu label aroma. Tiap classifier berisi banyak pohon. Keluar 109 skor; aturan ambang mengubah skor menjadi daftar label.

Model belajar dari pasangan fitur dan anotasi yang sudah tersedia. Jadi, ini prediksi anotasi aroma berdasarkan struktur, bukan alat yang mencium molekul secara langsung. Angka 90% pada tampilan adalah skor 0,90 yang dikali 100, bukan akurasi 90% dan belum terbukti sebagai peluang sensoris yang terkalibrasi.

Contoh nyata yang akan kita ikuti adalah vanillin: `COc1cc(C=O)ccc1O`. Vanillin sudah ada dalam dataset. Contoh ini membantu menelusuri alur dan tidak dipakai sebagai bukti kemampuan pada molekul baru.

Urutan belajar yang masuk akal

Satu sesi bisa 30-45 menit. Kalau masih bingung, ulangi satu sesi; nggak perlu mengejar semua bab dalam sehari.

Sesi	Buka materi	Setelah itu kamu bisa menjelaskan
1	01 Dasar dan penelitian terdahulu	Masalah penelitian dan hubungannya dengan penelitian acuan
2	02 Molekul, fitur, dan target	Perbedaan SMILES, X, y, skor dan label
3	03 Dataset dan sumber	Bentuk data sebelum dibersihkan dan cara menyambung identitas
4	04 Preprocessing	Baris ditolak, label dirapikan, molekul digabung
5	05 Grup, split dan fold	Kenapa data dibagi dan bagaimana mencegah irisan grup
6	06 Model dan proses belajar	109 classifier, boosting, loss dan bobot
7	07 Optuna dan validasi	Satu trial, CV, OOF dan training final
8	08 Evaluasi dan hasil	Arti metrik dan alasan pemilihan LightGBM D
9	09 Menjalankan dan prediksi	Membuka notebook serta memakai model tersimpan
10	10 Latihan sidang dan kamus	Menjelaskan alur tanpa membaca catatan

Kenapa teori dulu? Kalau langsung lihat `fit()` sebelum tahu apa itu X dan y, kode terasa seperti kumpulan perintah acak. Setelah tahu tujuan tiap langkah, kamu bisa menghubungkan baris kode dengan pekerjaan yang nyata.

Cara menggunakan folder ini

Folder `MATERI_BELAJAR/` berisi versi per bab dari panduan ini. Baca materi konsep dulu, lalu buka notebook yang disebutkan. Source asli ada di **KODE_ML/research/**. Notebook memanggil dan menampilkan fungsi dari source itu; kamu tidak sedang belajar algoritma pengganti.

Yang kamu cari	Buka dari KODE_ML
Urutan belajar dan progres	MULAI_BELAJAR.md dan CHECKLIST_BELAJAR.md
Panduan lengkap untuk dibaca	PANDUAN_BELAJAR_DARI_NOL.pdf
Penjelasan tiap bab	MATERI_BELAJAR/00 sampai 10
Praktik tahap demi tahap	Delapan notebook bernomor 01 sampai 08
Implementasi aktif	`research/*.py` dan `research/config/*.json`
Data mentah, bersih dan siap training	DATA_UNTUK_BELAJAR/README.md
Angka hasil dan parameter tersimpan	HASIL_UNTUK_BELAJAR/README.md
Paper acuan dan daftar bacaan	REFERENSI_BELAJAR/README.md
Cuplikan source dengan baris dan hash	MATERI_BELAJAR/11_Peta_Source_dan_Cuplikan.md

Nama notebook panjang `BELAJAR_KODE_ML_LENGKAP.ipynb` sudah menjadi arsip. Yang aktif sekarang delapan notebook. Urutannya untuk belajar, bukan kewajiban menjalankan notebook 01 dulu agar notebook 08 bisa berjalan.

Membaca kode tanpa tenggelam

Untuk satu fungsi, jawab empat hal: **inputnya apa, proses utamanya apa, outputnya apa, dan dipakai siapa setelahnya**. Baru baca baris per baris. Sel `tampilkan_sumber(...)` memberi lokasi file, baris dan kode aktual. Catat variabel yang belum kamu pahami; jangan menebak dari namanya saja.

File `belajar.py` cuma membantu menampilkan source. `audit_labels.py` memeriksa pemetaan label. `asal_kohor.py` menelusuri asal pembagian data historis. Ketiganya tidak berarti training sedang berjalan.

Patokan progres

Kamu boleh lanjut kalau bisa menjelaskan dengan kata sendiri, memberi satu contoh, dan menunjuk bukti di file. Kalau cuma bisa mengulang istilah seperti 'boosting' tanpa tahu yang dijumlahkan apa, kembali ke contoh kecilnya.

Latihan awal: ceritakan alur vanillin dalam lima kalimat. Jangan pakai kata 'akurasi' untuk satu skor aroma. Contoh jawaban: 'SMILES dibaca menjadi struktur. Struktur diubah menjadi fitur. Setiap classifier memberi skor untuk satu label. Skor dibandingkan dengan ambang. Beberapa label bisa lolos bersamaan.'

Pertanyaan dosen: 'Apa sebenarnya yang dipelajari model?' Jawaban singkat: 'Hubungan antara fitur struktur molekul dan anotasi aroma pada dataset. Prediksinya bergantung pada kualitas anotasi dan representasi fitur yang digunakan.'

Checklist: [ ] bisa cerita input-proses-output; [ ] tahu folder source; [ ] tahu hasil utama yang dipakai; [ ] bisa membedakan contoh dan hasil nyata.

1. Sebenarnya skripsi kita mau menjawab apa?

Tujuan belajar: kamu bisa menjelaskan masalah penelitian, bedanya belajar dan memprediksi, serta hubungan penelitianmu dengan penelitian sebelumnya.

1.1 Mulai dari satu pertanyaan

Pertanyaan kita kira-kira begini: **"Kalau kita tahu struktur satu molekul, label aroma apa saja yang kemungkinan cocok menurut pola data penelitian?"**

Contohnya **vanillin**. Di dataset gabungan kita, molekul ini punya catatan `caramel`, `chocolate`, `creamy`, `gourmand`, `powdery`, `sweet`, dan `vanilla`. Jadi kita butuh sistem yang bisa memberi **beberapa label sekaligus**. Istilahnya **multi-label**. `Sweet` boleh muncul bersama `vanilla`; sistem tidak dipaksa memilih salah satu.

Yang dipelajari adalah hubungan **struktur → label yang tercatat**. Sistem ini tidak mencium sampel, tidak membaca resep parfum, dan tidak mengukur kadar bahan. Struktur saja juga belum mencakup kondisi penciuman, konsentrasi, atau perbedaan persepsi tiap orang. Batas ini perlu kamu ingat saat menjelaskan manfaatnya.

1.2 Input, proses, output

Bagian	Yang terjadi dalam proyek kita
Input pengguna	Teks SMILES yang menyatakan struktur satu molekul
Pemeriksaan	RDKit mengecek apakah struktur bisa dibaca dan hanya satu komponen
Ubah menjadi angka	Buat Morgan 2.048 bit; pada kondisi B/D tambahkan delapan deskriptor
Model	Sekumpulan classifier XGBoost atau LightGBM memberi skor tiap label
Output	Sebanyak 109 skor, lalu bisa diurutkan atau dijadikan keputusan dengan ambang

Classifier artinya model untuk membedakan kelas. Untuk label `vanilla`, dua kelasnya adalah target **1** dan **0**.

Jangan tertukar dua kegiatan ini:

- **Training:** model melihat banyak contoh fitur beserta targetnya, lalu membentuk aturan pohon.
- **Prediksi:** model yang sudah tersimpan menerima fitur molekul, lalu menjalankan aturan pohonnya. Model tidak belajar ulang setiap pengguna memasukkan SMILES.

Saat training, nama aroma dipakai untuk menentukan target. Saat prediksi, model menerima **angka fitur struktur**, bukan bocoran label asli atau nama "vanillin".

1.3 Jadi, ada 109 model atau 109 pohon?

Ada **109 classifier biner untuk satu kandidat lengkap**, satu untuk setiap label. Pendekatan ini biasa disebut **Binary Relevance**: masalah besar dipecah menjadi pertanyaan kecil, misalnya "vanilla atau bukan?", "sweet atau bukan?", dan seterusnya.

Tetapi **satu classifier berisi banyak pohon**. XGBoost dan LightGBM adalah algoritma boosting: pohon ditambahkan bertahap supaya gabungan prediksinya makin sesuai target training. Jadi gambar yang tepat adalah:

```
Fitur molekul yang sama
  ├─ classifier vanilla → banyak pohon → satu skor vanilla
  ├─ classifier sweet   → banyak pohon → satu skor sweet
  └─ ... sampai 109 label
```

Cuplikan dipersingkat dari kode aslinya terlihat di training.py, fungsi fit_final_models, baris 55:

```
for j, name in enumerate(labels):
    # pemeriksaan model yang sudah tersimpan ada di antara baris ini
    model = fit(algorithm, params, x, y[:, j], Limits())
```

`j` adalah nomor label. `x` adalah fitur semua contoh training. `y[:, j]` mengambil **satu kolom target**, misalnya target `vanilla` untuk seluruh molekul. Jadi semua classifier melihat fitur struktur yang sama, tetapi pertanyaan targetnya berbeda.

Kalau membandingkan satu kandidat XGBoost dengan satu kandidat LightGBM, ada **109 × 2 = 218 classifier**. Kalau seluruh delapan kandidat A-D dari dua algoritma dihitung, ada **872 classifier final**. Jumlah tersebut belum menghitung model sementara selama validasi. Saat pengguna memprediksi dengan satu kandidat terpilih, yang dipakai satu set 109 classifier tersebut.

Batas Binary Relevance: model `vanilla` tidak menerima skor `sweet` sebagai input. Hubungan antarlabel tidak dipelajari secara eksplisit melalui rantai model.

1.4 Skor 80% itu maksudnya apa?

Contoh **ilustrasi**, bukan hasil eksperimen:

Label	Skor	Keputusan jika ambang 0,5
vanilla	0,80	Ya
sweet	0,70	Ya
citrus	0,10	Tidak

Angka `0,80` bisa ditampilkan sebagai `80%`, tetapi sebut **skor prediksi kelas positif**. Jangan menyebutnya “akurasi vanillin 80%”, “kadar vanilla 80%”, atau “pasti 80 dari 100 orang mencium vanilla”. Accuracy menghitung benar-salah pada sekumpulan data yang punya target pembanding. Skor satu molekul berbeda. Skor antarlabel juga **tidak harus berjumlah 100%**.

1.5 Penelitian terdahulu dipakai untuk apa?

Penelitian sebelumnya memberi dasar bahwa masalah ini bisa dimodelkan dan membantu menentukan pilihan yang layak diuji. Hasil mereka bukan skor baseline internal kita.

Studi primer	Hal yang relevan	Batas perbandingan
Saini dan Ramanathan, 2022	Menggabungkan 7.374 molekul dengan 109 label; menguji Binary Relevance dan Classifier Chains berbasis Random Forest. Menilai micro-F1.	Jumlah label yang sama tidak menjamin kosakata dan distribusinya sama. Micro-F1 berbeda dari macro AP kita.
Suh dkk., 2025	Menggunakan 8.681 molekul dari 10 sumber. Membandingkan fitur gugus fungsi, delapan deskriptor, dan Morgan secara terpisah pada RF/XGBoost/LightGBM. ST-XGB dilaporkan AUROC 0,828 dan AUPRC 0,237.	Proyek kita memakai lima sumber, 6.686 molekul, dan 109 target. Morgan + delapan deskriptor adalah rancangan eksperimen kita, bukan konfigurasi Suh yang disalin persis.

Alasan yang terdokumentasi di Bab 2: gabungan fitur, tuning, dan algoritma dibandingkan dengan baseline internal pada data penelitian yang sama. **Penjelasan untuk memahaminya:** Morgan merangkum pola lokal struktur, sedangkan delapan angka merangkum sifat molekul; masuk akal untuk menguji apakah keduanya saling membantu. Itu alasan menguji, belum bukti bahwa penambahan fitur pasti meningkatkan hasil.

Untuk dasar algoritmanya, rujukan primer adalah Chen dan Guestrin, XGBoost, 2016, Ke dkk., LightGBM, 2017, dan Akiba dkk., Optuna, 2019. Optuna adalah alat pencari pengaturan training; **bukan classifier ketiga** yang memprediksi aroma.

1.6 Urutan baca dan latihan

1. Baca SAFE_EXECUTION_PLAN.md untuk keputusan penelitian. Untuk status aplikasi, prioritaskan audit 27 Oktober di HASIL_UNTUK_BELAJAR/audit_27_oktober/README.md karena status ONNX pada plan belum diperbarui.
2. Baca Bab2.tex, terutama penelitian terdahulu dan multi-label. Ini bab yang dipanggil laporan utama `SKRIPSI.tex`.
3. Lihat 05_Model_dan_Bobot.ipynb setelah gambaran besar ini terasa masuk akal.

Latihan: kalau ada tiga aroma dan dua algoritma, berapa classifier untuk satu kondisi tiap algoritma? Jawaban: **enam**. Kalau tiap classifier punya 100 pohon, jumlah pohonnya 600. Ini hanya contoh.

Kalau dosen bertanya:

- **"Apa yang diprediksi?"** "Label aroma yang tercatat pada dataset berdasarkan fitur struktur satu molekul. Beberapa label bisa diprediksi bersamaan."
- **"Kenapa tidak langsung bilang lebih baik dari Suh?"** "Data, label, pembagian, dan perhitungan metriknya berbeda. Kesimpulan perbandingan saya didasarkan pada kandidat yang diuji dalam protokol penelitian saya."
- **"109 label berarti satu pohon per aroma?"** "Satu classifier per aroma, dan setiap classifier merupakan gabungan banyak pohon boosting."

Cek paham sebelum lanjut: kamu bisa membedakan training/prediksi, classifier/pohon, skor/accuracy, serta baseline internal/skor artikel lain tanpa membaca teks ini.

2. Dari SMILES menjadi fitur dan target

Tujuan belajar: kamu bisa menunjukkan bentuk satu molekul sebelum dan setelah diubah menjadi angka, menjelaskan delapan deskriptor, dan membedakan angka input dengan angka jawaban.

2.1 SMILES itu tulisan struktur

SMILES adalah cara menuliskan struktur molekul sebagai teks. Contoh pendek: `cco` menyatakan etanol. Huruf dan simbolnya menyatakan atom, ikatan, cabang, dan cincin. Dasarnya dibahas dalam Weininger, 1988.

Contoh yang kita ikuti adalah **vanillin**. Salinan data mentah GoodScents memiliki CID `1183`, nama `vanillin`, dan SMILES:

```
COC1=C(C=CC(=C1)C=O)O
```

Setelah dibaca RDKit dan ditulis dalam bentuk kanonik, catatan gabungan menggunakan:

```
COc1cc(C=O)ccc1O
```

Keduanya adalah penulisan struktur yang sama dalam contoh ini. **Kanonik** berarti perangkat lunak memilih penulisan yang konsisten. Jadi teks berubah tidak otomatis berarti molekulnya berubah. Di kode preprocessing, `Chem.MolToSmiles(mol, isomericSmiles=True)` mempertahankan informasi stereokimia yang tersedia; Morgan yang dipakai model kemudian dikonfigurasi tanpa kiralitas.

Lihat bukti aktualnya di molecules.csv GoodScents, baris 102 dan records.csv hasil preprocessing, baris 5739. Nomor `source_row` vanillin adalah **5737**; ini berbeda dari nomor baris teks karena ada header dan penomoran dimulai dari nol.

2.2 Library apa yang mengerjakan bagian ini?

Di features.py, baris 2:

```
import numpy as np
from rdkit import Chem
from rdkit.Chem import Descriptors, rdFingerprintGenerator
```

Nama	Kerjanya di file ini
NumPy, disingkat `np`	Menyimpan angka dalam array dan menggabungkan kolom fitur
RDKit Chem	Membaca SMILES menjadi objek molekul dan memeriksa komponennya
Descriptors	Menghitung delapan sifat molekul
rdFingerprintGenerator	Membuat generator Morgan dan menghasilkan fingerprint

`Chem.MolFromSmiles(smiles)` mengubah teks menjadi objek molekul. Fungsi `molecule()` pada baris 12-16 menolak input yang gagal dibaca atau terdiri atas lebih dari satu komponen. Ini aturan implementasi, jadi tidak perlu dipaksakan menjadi rumus.

2.3 Apa isi 2.048 angka Morgan?

Bayangkan struktur molekul sebagai jaringan: atom adalah titik dan ikatan adalah garis. Morgan memeriksa lingkungan sekitar setiap atom, lalu memperluasnya sampai **radius 2**, yaitu sampai dua langkah ikatan dari pusat atom. Identitas pola-pola itu dipetakan ke posisi dalam vektor biner. **Biner** berarti isinya 0 atau 1.

Kode aktual, features.py, fungsi calculate_morgan, baris 29:

```
generator = rdFingerprintGenerator.GetMorganGenerator(
    radius=RADIUS, fpSize=N_BITS, includeChirality=False)
return np.asarray([generator.GetFingerprintAsNumPy(molecule(smi, i))
                   for i, smi in enumerate(smiles)], dtype=np.uint8)
```

`RADIUS=2` dan `N_BITS=2048` ditetapkan pada baris 6-7. `includeChirality=False` berarti informasi kiralitas tidak dimasukkan ke fingerprint. Untuk satu SMILES, hasilnya satu baris dengan 2.048 kolom; untuk sepuluh SMILES, bentuknya `(10, 2048)`.

Contoh **ilustrasi delapan posisi**, bukan fingerprint vanillin asli:

```
[0, 1, 0, 0, 1, 0, 1, 0]
```

Angka 1 berarti ada pola yang terpetakan ke posisi tersebut. Angka 0 berarti posisi itu tidak aktif. **Bit ke-5 bukan "fitur aroma vanilla".** Posisi fingerprint mewakili hasil pemetaan pola struktur. Beberapa pola bisa bertabrakan di posisi yang sama; ini disebut *hash collision*. Karena itu satu bit juga tidak selalu berarti satu substruktur unik. Dasar circular fingerprint dijelaskan oleh Rogers dan Hahn, 2010; penggunaan Morgan di RDKit dijelaskan dalam dokumentasi resmi RDKit.

2.4 Delapan deskriptor itu apa?

Deskriptor adalah **angka ringkasan sifat struktur**. Nilai berikut berasal dari file hasil aktual, bukan contoh karangan: descriptors.csv, baris 5739.

Urutan	Nama kode	Cara membayangkannya	Vanillin
1	`MolWt`	Bobot molekul berdasarkan bobot atom rata-rata	152,149
2	`MolLogP`	Estimasi kecenderungan molekul berada pada lingkungan lemak dibanding air	1,2133
3	`NumHDonors`	Jumlah donor ikatan hidrogen menurut aturan RDKit	1
4	`NumHAcceptors`	Jumlah akseptor ikatan hidrogen menurut aturan RDKit	3
5	`TPSA`	Ringkasan luas permukaan polar, satuan $Å^2$	46,53
6	`NumRotatableBonds`	Jumlah ikatan yang dinilai bisa berotasi	2
7	`HeavyAtomCount`	Jumlah atom selain hidrogen	11
8	`RingCount`	Jumlah cincin menurut perhitungan RDKit	1

Kamu belum perlu menghitung TPSA atau logP manual. Kode memakai fungsi RDKit, bukan rumus buatan sendiri. Untuk sidang, pahami nama, arti, sumber perhitungan, dan urutannya.

Di `calculate()` baris 19-26, kode membuat tempat untuk `(jumlah molekul, 8)`, membaca tiap molekul, lalu menjalankan fungsi yang namanya ada di `DESCRIPTORS8`. `getattr(Descriptors, name)(mol)` berarti "ambil fungsi deskriptor bernama `name`, lalu hitung untuk `mol`". Nilai tak hingga atau tidak valid menyebabkan error; tidak diam-diam diganti angka lain.

Pada kondisi B/D, fungsi `features()` baris 41-45 menempelkan delapan angka itu setelah Morgan. Maka:

```
1 molekul: 2.048 bit Morgan + 8 angka deskriptor = 2.056 fitur
X training: 5.353 baris × 2.056 kolom
X test:     1.333 baris × 2.056 kolom
```

X adalah nama umum untuk matriks input. "Matriks" cukup dibayangkan sebagai tabel angka. Kondisi A/C memakai Morgan saja, sehingga 2.048 kolom. Urutan fitur saat prediksi harus sama dengan saat training.

2.5 Target 0/1 berbeda dari fitur 0/1

Sekarang pindah ke **jawaban yang ingin dipelajari**, yaitu target `Y`. Misalnya kita tampilkan empat dari 109 label:

Molekul	chocolate	sweet	vanilla	citrus
Vanillin	1	1	1	0

Dalam proyek ini **1 = label tercatat setelah pemetaan dan penggabungan sumber**. **0 = label tidak tercatat sebagai target positif**. Nilai 0 belum membuktikan bahwa semua orang pasti tidak akan mencium aroma itu. Data anotasi bisa tidak lengkap.

Rumus ringkasnya:

```
y(i,j) = 1 jika label j ada pada daftar label molekul i; selain itu 0.
```

`i` adalah nomor molekul, `j` nomor label. Ini persis aturan pada dataset.py, fungsi encode_frozen_records, baris 103:

```
target = np.asarray([[int(name in row) for name in meta["labels"]]
                     for row in records.labels], dtype=np.uint8)
```

`name in row` memeriksa keanggotaan label; `int(True)` menjadi 1 dan `int(False)` menjadi 0. Daftar serta urutan 109 label mengikuti registri yang sudah dibekukan. Fungsi audit aktif ini **tidak memilih ulang label**. Pengaturan asalnya mensyaratkan sedikitnya 30 positif pada training dan tidak menetapkan batas "ambil 109 teratas".

Fitur menjawab **"strukturnya seperti apa?"**, target menjawab **"label apa yang dicatat untuk contoh ini?"**. Bentuk target training adalah `(5353, 109)`. Banyak angka 1 dalam satu baris diperbolehkan.

2.6 Urutan praktik dan persiapan sidang

1. Ikuti vanillin pada CSV mentah dan `records.csv` yang ditautkan di atas.
2. Baca 03_Dataset_dan_Fitur.ipynb sambil melihat `features.py`.
3. Buka CSV deskriptor dan `target_train_109_label.csv` dalam folder siap training. Keduanya punya `source_row` agar molekulnya bisa dicocokkan. CSV target memiliki kolom identitas tambahan; **109** mengacu pada kolom target aromanya.

Latihan: tiga molekul memakai kondisi D. Berapa ukuran `X` dan `Y`? Jawaban: `X=(3,2056)` **dan** `Y=(3,109)`. Jika satu molekul punya tiga label tercatat, baris targetnya mengandung tiga angka 1, bukan satu angka 3.

Kalau dosen bertanya:

- **"Kenapa SMILES tidak langsung masuk ke XGBoost?"** "Classifier menerima fitur numerik. RDKit mengubah struktur SMILES menjadi Morgan dan deskriptor dengan pengaturan yang tetap."
- **"Delapan tambahannya juga bit?"** "Bukan. Delapan deskriptor adalah angka sifat molekul; beberapa berupa hitungan dan beberapa pecahan."
- **"Target nol berarti pasti tidak beraroma itu?"** "Nol berarti label tersebut tidak tercatat sebagai positif dalam dataset. Keterbatasan kelengkapan anotasi tetap ada."

Cek paham sebelum lanjut: tunjuk satu baris vanillin, jelaskan kenapa ada 2.056 fitur tetapi 109 target, dan bedakan bit Morgan dengan target aroma tanpa menyebut keduanya sebagai hal yang sama.

03. Dataset awal bentuknya seperti apa?

Tujuan dan file yang dibuka

Di akhir bab ini kamu bisa memperlihatkan data sebelum preprocessing dan menjelaskan bagaimana satu anotasi aroma ditemukan SMILES-nya. Mulai dari notebook **01_Sumber_Data.ipynb** dan folder **DATA_UNTUK_BELAJAR/01_SEBELUM_PREPROCESSING/**.

Jangan membayangkan dataset awal sudah berupa satu tabel rapi berisi 2056 angka dan 109 target. Snapshot awal punya **15 CSV dari lima sumber**. Tiap sumber punya tabel molekul, tabel penghubung stimulus, dan tabel anotasi aroma.

Tabel	Isinya	Perannya
molecules.csv	Identitas molekul dan SMILES	Mengetahui struktur molekul
stimuli.csv	Stimulus dan penghubung CID/CAS	Menyambungkan anotasi ke molekul
behavior.csv	Anotasi aroma	Mengetahui label yang dicatat sumber

Arctander memakai nama `behavior_1_sparse.csv`. Nama kolom tidak seragam, jadi cara membaca anotasinya mengikuti sumbernya. GoodScents memakai teks yang dipisahkan titik koma; IFRA punya tiga kolom descriptor; Leffingwell memakai kolom 0/1; Sigma memakai nilai True/False. Aturan ini benar-benar ada di `research/preprocessing.py`, fungsi `tokens_for`.

Mengapa ada penghubung identitas?

Misalnya tabel aroma cuma menulis Stimulus `121-33-5`. Kode perlu tahu stimulus itu menunjuk molekul yang mana, lalu mengambil SMILES-nya. CID adalah ID molekul PubChem; CAS adalah nomor registrasi zat; Stimulus adalah ID sampel dalam sumber. Ketiganya bukan fitur yang dipelajari model.

Contoh nyata GoodScents untuk vanillin:

Langkah	Nilai dalam data
behavior	Stimulus 121-33-5; sweet;vanilla;creamy;chocolate
stimuli	Stimulus 121-33-5 terhubung ke CID 1183
molecules	CID 1183, vanillin, COC1=C(C=CC(=C1)C=O)O
Sesudah digabung	Anotasi dan SMILES berada pada baris yang sama

Untuk mencarinya, buka CSV GoodScents lalu cari CID `1183` di molecules dan `121-33-5` di stimuli/behavior. Bisa dibuka sebagai teks atau CSV; ini belum tabel fitur training.

Kode asli: menyambungkan tabel

Buka `research/source_identity.py`, fungsi `_attach_smiles`. Potongan berikut benar-benar bagian implementasinya:

```
frame = beh.merge(bridge, on="Stimulus", how="left", validate="many_to_one")
return frame.merge(lookup, on="__cid__", how="left", validate="many_to_one")
```

`beh` adalah tabel anotasi. `bridge` menghubungkan Stimulus ke CID. `lookup` menghubungkan CID ke SMILES. `merge` berarti menyambungkan tabel memakai kunci yang sama. `left` mempertahankan baris anotasi; identitas yang gagal ditemukan tetap terlihat supaya bisa diaudit. `many_to_one` memastikan banyak baris anotasi tidak tersambung ke beberapa identitas yang bertentangan.

Sigma punya jalur khusus `_attach_sigma` dan `_cas_bridge`: alias CAS harus menunjuk satu identitas yang bisa dipastikan dari sumber lain. Kode tidak asal memilih salah satu saat identitasnya konflik. Kasus yang tidak terselesaikan dicatat dan dikeluarkan dari data yang diterima.

Snapshot dan config itu untuk apa?

`research/config/sources.json` mencatat lima sumber, nama CSV, commit snapshot, aturan dukungan label dan split. Commit adalah penanda versi data. `load_sources()` membaca snapshot lokal dan membandingkan SHA256 file dengan manifest. SHA256 bisa dibayangkan sebagai sidik jari isi file: perubahan isi akan membuat nilainya berubah.

Ini membantu memastikan data yang diperiksa sama dengan arsip penelitian. Tidak ada unduhan otomatis ketika notebook belajar membaca snapshot. Konfigurasi fitur model saat ini ada di `research/config/experiment.json`; jangan menyamakan seluruh config dengan pengaturan fingerprint.

Alur jumlah data yang benar

Tahap	Jumlah	Satuan
Baris anotasi awal	13.337	Baris sumber sebelum cleaning
Baris diterima	11.777	Baris yang lolos aturan cleaning
Baris ditolak	1.560	Baris dengan alasan penolakan tercatat
Sesudah molekul yang sama digabung	6.686	Molekul unik dalam kohor

13.337 bukan jumlah molekul unik. Satu molekul bisa muncul di beberapa sumber atau beberapa catatan. Penggabungan dilakukan berdasarkan SMILES canonical yang masih mempertahankan informasi stereo.

Ini bukti yang dipakai: `02_SETELAH_PREPROCESSING/audit.json`, `records.csv` dan `provenance.json`. Provenance menjawab 'baris bersih ini berasal dari sumber/baris yang mana?'. Mengetahui asal data membantu kamu menjawab dosen tanpa menyebut hasil gabungan sebagai data mentah.

Latihan dan pertanyaan sidang

Latihan: buka tiga CSV GoodScents. Cari vanillin dan tulis urutan Stimulus -> CID -> SMILES. Setelah itu buka records.csv dan cari canonical SMILES vanillin. Jangan mengharapkan teks SMILES mentah selalu identik karena penulisannya sudah dikanonisasi.

Pertanyaan: 'Apakah datanya hasil mencium parfum sendiri?' Jawaban: 'Tidak. Penelitian memakai anotasi dari lima sumber Pyrfume. Yang diprediksi adalah beberapa label untuk satu molekul berdasarkan struktur, bukan aroma campuran parfum atau hasil uji sensoris sendiri.'

Pertanyaan: 'Mengapa tidak langsung gabung dari nama senyawa?' Jawaban: 'Nama bisa memiliki sinonim atau penulisan berbeda. Implementasi menggunakan penghubung Stimulus, CID dan CAS yang diperiksa konsistensinya untuk mendapatkan SMILES.'

Pertanyaan: 'Lima sumber berarti semuanya bahan parfum?' Jawaban: 'Tidak otomatis. Lima sumber menyediakan anotasi molekul; cakupan penggunaannya perlu dibatasi dan tidak membuktikan semua molekul hanya dipakai dalam parfum.'

Checklist: [ ] bisa menunjukkan tiga jenis CSV; [ ] bisa menelusuri vanillin; [ ] membedakan baris anotasi dan molekul unik; [ ] tahu kenapa identitas bisa ditolak.

04. Dari catatan sumber menjadi data bersih

Tujuan dan urutan proses

Buka notebook **02_Preprocessing_dan_Audit_Label.ipynb**, `research/preprocessing.py` dan **DATA_UNTUK_BELAJAR/02_SETELAH_PREPROCESSING/**. Targetnya: kamu bisa menjelaskan apa yang dibuang, apa yang dirapikan, dan apa yang tetap menjadi batasan data.

`merge_sources()` menerima tabel lima sumber. Untuk setiap baris: cari SMILES, periksa molekul, ambil istilah aroma sesuai format sumber, petakan ke istilah IFRA, lalu gabungkan anotasi untuk molekul yang sama. Ini aturan pengolahan data, jadi tidak perlu dipaksakan menjadi rumus ML.

Pemeriksaan struktur

Potongan asli di `merge_sources()`:

```
mol = Chem.MolFromSmiles(smiles)
if mol is None or mol.GetNumAtoms() == 0:
    reason = "invalid_smiles"
elif len(Chem.GetMolFrags(mol)) != 1:
    reason = "multifragment"
```

`Chem` berasal dari RDKit. `MolFromSmiles` mencoba membaca teks menjadi struktur. `None` berarti pembacaan gagal. `GetMolFrags` menghitung komponen yang terpisah. Data dengan beberapa komponen dikeluarkan karena penelitian dibatasi pada satu molekul terhubung; ini bukan model untuk campuran aroma.

Kalau identitasnya hilang, baris ditolak sebelum blok ini. Alasan penolakan disimpan di `rejected_records.json`, jadi bukan dibuang tanpa jejak.

Alasan penolakan	Jumlah baris aktual
Identitas tidak tersedia	482
Beberapa fragmen	313
Tidak ada istilah label	213
Ada istilah, tidak ada yang terpetakan	552
SMILES invalid / behavior invalid	0 / 0
Total	1.560

Ini hitungan baris sumber, bukan 1.560 molekul unik. Total 482 + 313 + 213 + 552 = 1.560.

Label dirapikan, bukan diprediksi saat cleaning

`normalize()` di `source_identity.py` mengubah huruf menjadi kecil, merapikan tanda baca dan spasi. Contoh ilustrasi: `Sweet Notes` menjadi `sweet notes`. `map_labels()` lalu mencoba mencocokkan istilah dengan taksonomi IFRA.

```
term = normalize(token)
mapped = term if term in taxonomy else ALIASES.get(term)
```

`token` adalah satu istilah dari sumber. `taxonomy` adalah daftar istilah acuan. Kalau istilahnya sudah ada, langsung dipakai. Kalau belum, kode mencari aturan di `ALIASES`. Contoh aturan aktual: `fruit -> fruity`, `cream -> creamy`, `powder -> powdery`. Istilah yang tetap gagal tidak ditebak dengan fuzzy matching.

Bedakan tiga angka ini: **31 aturan alias**, **184 istilah IFRA**, dan **109 target model**. 31 bukan jumlah label. 184 adalah daftar kandidat istilah hasil pembacaan IFRA. 109 baru didapat setelah seleksi dukungan pada data training.

Baris diterima juga bisa kehilangan sebagian label

Misalnya secara ilustrasi satu baris punya `sweet` dan istilah yang tidak dikenali. Baris masih diterima karena `sweet` berhasil dipetakan, tetapi istilah lain tidak masuk target. Jadi 'baris diterima' tidak berarti seluruh informasi labelnya ikut.

Audit aktual `hasil_audit_label/summary.json` mencatat **45.689 kemunculan istilah** yang mencapai mapper: 37.940 exact, 1.255 alias, 6.494 tidak terpetakan. Istilah gagal itu terdiri dari 498 bentuk berbeda. Ada **3.898 baris diterima** yang sebagian istilahnya tidak terpetakan. Itu berbeda dari 552 baris yang ditolak karena tidak ada satu pun label terpetakan.

Ada juga 1.230 kemunculan istilah yang berhasil dipetakan tetapi labelnya tidak masuk 109 target karena dukungan training tidak cukup. Jangan mencampur kegagalan mapping dengan seleksi target. Lihat `unmatched_terms.csv` dan `taxonomy_selection.csv` untuk mengecek rincian.

Aturan alias sudah diperiksa konsistensi eksekusinya, tetapi kesetaraan makna seluruh istilah belum divalidasi pakar aroma. Ini batas penelitian yang perlu kamu sampaikan dengan jujur.

Molekul yang sama digabung

Potongan asli berikut memakai himpunan, jadi label yang berulang tidak ditambahkan dua kali:

```
canonical = Chem.MolToSmiles(mol, isomericSmiles=True)
merged[canonical].update(labels)
origins[canonical].add(source)
```

`canonical` adalah penulisan SMILES standar dari RDKit. `merged` menyimpan gabungan label; `origins` menyimpan asal sumber. Hasilnya **6.686 baris**, satu baris per canonical SMILES. Vanillin di records.csv, `source_row` 5737, memiliki gabungan label: caramel, chocolate, creamy, gourmand, powdery, sweet, vanilla. Daftar ini lebih panjang daripada satu catatan GoodScents karena informasi beberapa sumber digabung.

Target 0/1 dibentuk setelah pemilihan label

Protokol memilih label dengan sedikitnya **30 positif dan 2 negatif pada train**. Aturan ini ada di `asal_kohor.py`, fungsi `encode_and_split`; registry-nya sekarang dibekukan. Dari 184 kandidat, tersisa 109. Angka 30 adalah keputusan protokol, bukan jaminan statistik bahwa semua label pasti bagus.

Dalam kode `encode_frozen_records`, target dibentuk dengan memeriksa keanggotaan label:

```
target = np.asarray([[int(name in row) for name in meta["labels"]]
                     for row in records.labels], dtype=np.uint8)
```

`name` adalah nama label, `row` kumpulan anotasi satu molekul. Ada `-> True` -> 1; tidak ada `-> False` -> 0. Urutan kolom mengikuti registry label, bukan urutan skor tertinggi. Bentuk seluruh target adalah **(6686, 109)**.

Nol berarti anotasi label tidak tercatat setelah pengolahan, bukan bukti sensoris bahwa aroma tersebut mustahil ada. Setelah hanya memakai 109 label, ada 11 baris train dan 4 test yang semua targetnya nol. Keduanya dipertahankan sesuai data beku.

Memastikan notebook memang memeriksa data nyata

`review_frozen_preprocessing()` mengulang cleaning di memori, memanggil `verify()`, lalu membandingkan hasil encoding dengan artefak beku. Ia tidak menimpa dataset atau membuat split baru. Inputnya snapshot + config; outputnya records, target, label, split dan tabel audit untuk dipelajari.

Latihan: pilih satu baris `label_mappings.csv` dengan method alias dan satu baris `unmapped_descriptors.csv`. Jelaskan kenapa hasilnya berbeda. Lalu buka `target_train_109_label.csv` dan tunjukkan satu angka 1 serta satu angka 0.

Pertanyaan dosen: 'Apakah label nol berarti molekulnya tidak punya aroma itu?' Jawaban: 'Dalam dataset, nol berarti label itu tidak tercatat pada target terpilih. Karena anotasi tidak lengkap dan ada harmonisasi istilah, saya tidak menyamakan nol dengan kepastian ketiadaan aroma.'

Pertanyaan: 'Mengapa cleaning perlu diaudit?' Jawaban: 'Cleaning menentukan molekul dan label yang dipelajari model. Saya menyimpan alasan penolakan, asal baris dan hasil mapping supaya perubahan jumlah data dapat ditelusuri.'

Checklist: [ ] bisa menjumlahkan alasan penolakan; [ ] memahami exact/alias/unmapped; [ ] membedakan 31/184/109; [ ] memahami arti target nol.

05. Mana yang dipakai belajar dan mana yang dipakai menguji?

Tujuan dan file yang dibuka

Buka notebook **04_Grup_Split_dan_Fold.ipynb**, `research/splits.py`, `asal_kohor.py` dan `DATA_UNTUK_BELAJAR/02_SETELAH_PREPROCESSING/pembagian_train_test.csv`. Kamu perlu bisa membedakan train, validation, test, fold dan seed sebelum membaca Optuna.

Istilah	Arti dalam penelitian ini
Train	5.353 molekul untuk mengembangkan resep dan melatih model
Validation	Bagian train yang sementara tidak dipakai fit dalam satu fold
Test	1.333 molekul pada partisi beku untuk evaluasi akhir protokol
Fold	Satu giliran pembagian fold-train dan fold-validation
Seed	Angka untuk membuat pengacakan bisa diulang
Group	Kumpulan molekul yang harus tetap di partisi yang sama

Jangan menyebut semua 5.353 molekul selalu dipakai fit setiap penilaian. Dalam CV, sebagian menjadi validation. Sesudah resep dan pemilihan kandidat dibekukan, model final kandidat dilatih memakai seluruh train.

Mengapa pembagiannya memakai grup?

Bayangkan dua baris sebenarnya mewakili struktur tanpa stereo yang sama. Kalau satu masuk train dan satu test, penilaian bisa terlalu mudah. Protokol menyatukan molekul ke grup jika strukturnya sama setelah stereo dihapus **atau** fingerprint Morgan historisnya identik. Hubungan ini diteruskan: kalau A tersambung B dan B tersambung C, ketiganya satu grup.

Batasnya: aturan ini menjaga identitas yang ditentukan protokol, bukan menjamin seluruh molekul mirip atau satu scaffold selalu bersama. Jadi jangan mengatakan 'semua kemiripan kimia sudah bebas kebocoran'.

Mengapa masih ada angka 1024 di asal grup?

Fitur model sekarang tetap 2048+8. Tetapi **identitas grup dan pembagian molekul sudah dibuat pada sejarah kohor memakai struktur tanpa stereo atau Morgan1024 identik**. Penelitian utama mempertahankan pembagian itu agar hasilnya sesuai artefak beku. `asal_kohor.py` menerima fingerprint historis tersimpan untuk memeriksa asal grup; ia tidak membuat fitur model 1024 untuk training utama.

Ini perlu diketahui agar penjelasanmu sesuai kode. Menghapus jejaknya tidak mengubah sejarah pembagian data. Mengelompokkan ulang dari 2048 berarti mengubah eksperimen. Karena itu materi menempatkan informasi ini di asal split, tidak menyebut 1024+5 sebagai model utama.

Outer split: train dan test

Kode asli `outer_split()` di `asal_kohor.py`:

```
cv = GroupShuffleSplit(n_splits=1, test_size=test_size, random_state=seed)
a, b = next(cv.split(np.zeros(len(groups)), groups=groups))
return np.sort(a), np.sort(b)
```

Input `groups` berisi nomor grup setiap molekul. `test_size=0.2` berlaku pada **jumlah grup**, jadi jumlah molekul test tidak harus tepat 20%. `seed=42` membuat pilihan grup dapat diulang. `a` dan `b` adalah indeks baris, bukan isi fitur. `np.sort` merapikan urutan indeks. Hasil aktualnya 5.353 train dan 1.333 test.

Seleksi 109 label memakai dukungan pada train setelah split ini. Jadi bukan memilih label berdasarkan label mana yang menghasilkan skor test bagus. Di jalur saat ini, `dataset.verify()` memeriksa bahwa indeks, urutan, hash dan label masih sesuai kontrak beku.

Inner fold: latihan penilaian di dalam train

`inner_folds()` di `research/splits.py` menggunakan `StratifiedGroupKFold`. Anggota satu grup wajib tetap bersama; stratified berusaha menjaga pembagian kelas 0/1 untuk label yang sedang dinilai. Setiap classifier memiliki target biner sendiri, sehingga **fold dibentuk per label dan per seed**. Jangan menganggap satu daftar fold universal dipakai untuk seluruh 109 label.

```
cv = StratifiedGroupKFold(n_splits=k, shuffle=True, random_state=seed)
candidates = list(cv.split(np.zeros(len(train)), truth, train_groups))
```

`truth` adalah target satu label, bukan matriks 109 label. `train_groups` adalah grup pada partisi train. `candidates` berisi pasangan indeks fold-train dan fold-validation. Kode memeriksa kedua kelas tersedia; kalau jumlah fold tidak layak bisa diturunkan karena kelayakan kelas, bukan karena skornya jelek. Pada hasil utama semua pasangan label-seed mendapat **5 fold**.

Contoh ilustrasi lima fold: giliran 1 menyimpan kelompok bagian 1 untuk validation dan belajar dari bagian 2-5. Giliran berikutnya menyimpan bagian 2. Setelah lima giliran, tiap molekul train pernah diprediksi saat tidak dipakai fit untuk prediksi itu. Inilah asal skor OOF, dibahas lebih lanjut di bab 07.

Lima seed evaluasi: **7, 21, 42, 84, 100**. Ini mengubah pembagian fold. Seed estimator model tetap 42 dalam implementasi utama. Jadi 'lima seed' tidak berarti ada lima test set baru atau semua komponen random diganti.

Pemeriksaan hasil pembagian

`validate_grouped_split()` memeriksa indeks tidak dobel, cakupan lengkap dan tidak ada grup melintasi train-test maupun fold-train/validation. Audit terbaru memeriksa 109 x 5 x 5 = **2.725 fold label-seed**; pembagiannya sama untuk delapan kandidat. Ini membantu perbandingan karena model diuji pada pembagian yang sama.

Tetap ada batasan: test pernah dilihat pada pengembangan sebelumnya. CV yang diulang juga bukan nested CV, karena tidak memakai lingkar validasi terpisah untuk mengukur keseluruhan proses pemilihan

hyperparameter. Jadi hasilnya dilaporkan sebagai penelitian eksploratori, bukan perkiraan generalisasi yang sepenuhnya bebas pengaruh pengembangan.

Latihan: pilih satu source_row di pembagian_train_test.csv. Lihat split dan group_id. Cari semua anggota grupnya dan cek apakah split-nya sama. Jangan melakukan pembagian ulang.

Pertanyaan dosen: 'Kenapa seed bukan jumlah pengulangan model?' Jawaban: 'Seed adalah pengendali pengacakan. Protokol memakai lima nilai seed untuk mengulang pembagian CV; setiap pengulangan tetap berisi lima fold per label.'

Pertanyaan: 'Apakah group split menjamin tidak ada kebocoran?' Jawaban: 'Pemeriksaan menunjukkan tidak ada irisan grup menurut aturan struktur/fingerprint yang digunakan. Saya tidak menyatakan semua bentuk kemiripan molekul atau seluruh risiko bias sudah hilang.'

Checklist: [ ] bisa menggambar train-test; [ ] bisa menjelaskan validation di dalam train; [ ] tahu fold per label; [ ] tahu asal grup historis; [ ] bisa menyebut batas penilaian.

6 Model dan proses belajar

Yang perlu kamu pahami

Targetnya: paham apa yang dipelajari model dan bedanya classifier dengan pohon.

Model tidak membaca tulisan SMILES langsung. RDKit lebih dulu mengubah molekul menjadi angka. Untuk skenario B dan D, satu molekul menjadi **2.056 angka**: Morgan 2.048 bit ditambah delapan deskriptor. Angka inilah yang masuk ke model.

Satu label punya satu model sendiri

Bayangkan tiga aroma: sweet, woody, floral. Target `[1, 0, 1]` berarti sweet dan floral tercatat, woody tidak. Setiap model belajar dari kolom aromanya sendiri.

Dalam project ini ada **109 label**. Jadi satu kandidat mempunyai **109 classifier biner**. Biner berarti jawabannya dua kelas: 0 atau 1. Setiap classifier berisi banyak pohon. Jadi **109 model bukan 109 pohon**.

Yang dihitung	Jumlah dan artinya
Label per kandidat	109 aroma yang diprediksi sendiri-sendiri
Kandidat eksperimen	2 algoritma × 4 skenario = 8 kandidat
Model final tersimpan	8 × 109 = **872 classifier**
Model yang dipakai prediksi utama	109 classifier milik kandidat terpilih

Angka 872 hanya model final. Model sementara saat tuning dan validasi jauh lebih banyak. Prediksi normal tidak menjalankan seluruh delapan kandidat sekaligus. Hasil utama memilih **lgbm_D**.

Mulai baca kode dari sini

Sumber aktif: `KODE_ML/research/training.py`. Buka juga notebook `05_Model_dan_Bobot.ipynb` sebagai teman membaca. Library **XGBoost** menyediakan `XGBClassifier`; **LightGBM** menyediakan `LGBMClassifier`. Kita mengatur dan memanggilnya, sedangkan pencarian percabangan pohon dilakukan di dalam library.

Potongan asli `make_model`, baris 9-18:

```
def make_model(algorithm, parameters):
    params = dict(parameters)
    profile = compute_profile()
    threads = profile["threads_per_model"]
    if algorithm == "xgb":
        return XGBClassifier(n_jobs=threads, random_state=42, eval_metric="logloss",
                             tree_method=profile["xgboost_tree_method"], **params)
    if algorithm == "lgbm":
        return LGBMClassifier(n_jobs=threads, random_state=42, verbosity=-1, **params)
    raise ValueError("Unknown algorithm.")
```

Urutannya: salin pengaturan, baca profil komputasi, ambil jumlah thread, lalu buat jenis classifier yang diminta. **Thread** adalah pekerja komputasi dalam satu model. Profil saat ini memakai CPU dan enam thread per model; model dijalankan bergantian. `random_state=42` menjaga pengacakan model tetap terkendali. `**params` memasukkan pengaturan seperti jumlah pohon dan learning rate. Fungsi ini baru membuat objek; model **belum belajar**.

Kemudian baca `fit`, baris 33-53. Pemanggilan `model.fit(...)` di baris 45 atau 51 adalah saat model belajar memakai fitur, target, dan bobot data.

Terakhir lihat `fit_final_models`, baris 55-66. Bagian pentingnya, baris 62-66:

```
        model = fit(algorithm, params, x, y[:, j], Limits())
        filename = f"label_{j:03d}.joblib"
        joblib.dump(model, folder / filename)
        saved[name] = {"file": filename, "sha256": digest(folder / filename)}
        write(folder / "models.json", saved)
```

`y[:, j]` mengambil kolom aroma ke-`j`. **joblib** menyimpan classifier. `sha256` memeriksa perubahan isi file. `models.json` menghubungkan aroma dengan file modelnya.

Input final fit: `x` berukuran `(5353, 2048)` untuk A/C atau `(5353, 2056)` untuk B/D; `y` berukuran `(5353, 109)`. Satu pemanggilan `fit` hanya menerima satu target `(5353,)`. **Output:** satu classifier per label yang disimpan sebagai `label_000.joblib`, dan seterusnya.

Pohonnya belajar apa

Pohon mempelajari aturan seperti “apakah bit ini aktif?” atau “apakah MolLogP melebihi batas tertentu?”. Batasnya dicari saat training, bukan aturan aroma yang kita tulis manual.

Boosting menambah pohon secara bertahap. Pohon berikutnya memperbaiki kekurangan gabungan pohon sebelumnya. Skor pohon dijumlahkan, lalu diubah menjadi skor kelas positif. Konsep penjumlahan pohon ini dijelaskan dalam panduan resmi XGBoost.

Untuk konfigurasi kita, XGBoost memakai `hist` dan kebijakan pertumbuhan default `depthwise`: percabangan lebih dekat akar didahulukan. LightGBM tumbuh `leaf-wise`: daun dengan potensi perbaikan terbesar didahulukan. Keduanya memakai histogram pada pengaturan ini. Jadi jangan bilang hanya LightGBM yang memakai histogram. Sumber: parameter XGBoost dan fitur LightGBM 4.6.0.

Hitungan kecil untuk membayangkan boosting

Ini ilustrasi buatan, bukan hasil eksperimen atau parameter project. Anggap untuk label sweet skor awal `z = 0`. Skor `z` belum berbentuk peluang. Ubah dengan:

```
p = 1 / (1 + exp(-z))
```

`p` adalah skor positif, sedangkan `exp` adalah fungsi eksponensial. Saat `z = 0`, `p = 0,5`. Misalkan pohon baru memberi koreksi `+2` dan learning rate `0,5`. Learning rate mengatur seberapa besar koreksi dipakai.

```
z_baru = z_lama + learning_rate × koreksi = 0 + 0,5 × 2 = 1
```

```
p_baru = 1 / (1 + exp(-1)) ≈ 0,731
```

Skor sweet naik menjadi 0,731. Koreksi `+2` diberikan untuk contoh; aslinya dipelajari library. Skor pohon tidak dirata-ratakan sebagai persentase.

Kenapa ada bobot kelas

Lihat `balanced_sample_weights`, baris 21-30. Rumus yang **benar-benar dipakai kode** adalah `bobot_kelas = N / (2 × jumlah_kelas)`. `N` berarti jumlah baris pada bagian latih saat itu.

Contoh buatan: sepuluh baris, delapan negatif dan dua positif. Bobot negatif `10/(2×8) = 0,625`; positif `10/(2×2) = 2,5`. Total bobot kedua kelas sama-sama lima. Ini membantu kelas positif yang jarang tercatat ikut diperhatikan. Baris tidak digandakan dan label tidak diubah. Saat CV, bobot dihitung dari **train fold**, bukan validation atau test.

Batasnya: bobot tidak menambah pengetahuan baru. Label 0 juga berarti tidak tercatat positif dalam dataset; ini belum tentu pembuktian bahwa manusia tidak akan mencium aroma tersebut.

Bekal sidang

“Apakah satu pohon untuk satu aroma?” “Satu aroma punya satu classifier. Setiap classifier berisi banyak pohon boosting. Seluruh 109 classifier memberi skor aromanya masing-masing.”

“Kenapa memakai bobot kelas?” “Sebagian aroma jarang muncul. Bobot dihitung hanya dari data latih agar kelas positif tidak kalah hanya karena jumlah negatif jauh lebih banyak.”

Cek pemahaman

- [ ] Aku bisa membedakan satu kandidat, satu classifier, dan satu pohon.
- [ ] Aku bisa menunjukkan `fit` sebagai proses belajar.
- [ ] Aku bisa menghitung contoh bobot kelas.
- [ ] Aku tidak menyebut skor prediksi sebagai akurasi atau intensitas aroma.

7 Optuna validasi dan training final

Yang perlu kamu pahami

Targetnya: bisa membedakan tuning, validasi, dan final fit.

Optuna adalah library pencari pengaturan model. **Trial** berarti satu percobaan satu paket pengaturan. Contohnya jumlah pohon, learning rate, dan kedalaman pohon. Dalam project ini, satu trial mencoba **satu paket pengaturan bersama untuk 109 label**, bukan 109 paket pengaturan terpisah.

Urutan kerja penelitian

Tahap	Apa yang dilakukan	Hasil yang diteruskan
Skenario A/B	Baseline n_estimators=100 (target 100 iterasi boosting), tanpa Optuna	Resep baseline
Skenario C/D	Mencoba resep lewat Optuna	Resep terbaik berdasarkan validasi
Repeated grouped CV	Mengecek delapan kandidat dengan lima seed	Nilai validasi dan prediksi OOF
Pembekuan keputusan	Mengunci resep, threshold, dan pilihan kandidat	`selection_frozen.json`
Final fit	Belajar ulang pada seluruh 5.353 baris train	109 model final tiap kandidat
Test	Memakai 1.333 baris test yang ditetapkan	Hasil evaluasi akhir

A/C memakai Morgan 2.048 saja; B/D memakai Morgan 2.048 ditambah delapan deskriptor. Empat study adalah xgb_C, xgb_D, lgbm_C, dan lgbm_D. Masing-masing punya anggaran aktif delapan jam: total anggaran tuning 32 jam. Ini bukan total waktu seluruh penelitian, karena CV dan final fit juga membutuhkan waktu.

Baca satu trial dahulu

Sumber: `KODE_ML/research/tuning.py`, fungsi `tune`, mulai baris 72. Potongan asli baris 98-105:

```
                params=suggest(trial,algorithm,protocol)
                trial.set_user_attr('resolved_parameters',params)
                atomic_dump(study.sampler,folder/'sampler.joblib')
                event(run,'trial_started',candidate=key,trial=trial.number)
                values=cross_validate(algorithm,params,x,y,meta['labels'],split['folds'],
positions,limits,meta['summary_labels'])
                score=summarize(values,meta['summary_labels'])['metrics']['auprc']
                study.tell(trial,score)
                event(run,'trial_completed',candidate=key,trial=trial.number,macro_ap=score,
best_macro_ap=study.best_value)
```

`suggest` memilih pengaturan; `cross_validate` melatih dan memvalidasi; `summarize` merangkum label; `study.tell` mengirim skor ke Optuna. **Skor lebih besar lebih baik.** TPE memakai hasil trial selesai untuk usulan berikutnya. Fase startup berlangsung sampai sepuluh trial selesai: resep awal diantrekan, usulan sisanya masih acak. Trial terbaik XGBoost adalah trial 2, masih fase awal. Jadi jangan mengklaim TPE adaptif terbukti mengungguli pencarian acak.

Nama kunci `auprc` di sini dihitung sebagai **Average Precision atau AP**, bukan integrasi trapezoid kurva. Objective trial adalah rata-rata AP antar-fold untuk setiap label, lalu rata-rata 109 label. Nilai logloss tidak menjadi skor pemilihan trial. Logloss berkaitan dengan kesalahan probabilitas ketika classifier belajar; AP dipakai di luar classifier untuk menilai kualitas peringkat hasilnya. Protokol tidak memakai early stopping atau pruning trial.

Input: matriks fitur train `(5353, 2048)` atau `(5353, 2056)`, target `(5353, 109)`, daftar fold, dan batas komputasi. **Output:** satu angka objective per trial; akhirnya satu resep terbaik per study. Trial yang terputus karena anggaran habis ditandai gagal. Waktunya tetap dihitung.

Fold itu apa dan apa yang terjadi di kode

Fold adalah bagian data untuk latihan dan pengecekan bergantian. Pada satu putaran, sebagian baris menjadi train fold dan sebagian menjadi validation fold. Model hanya belajar dari train fold, lalu menebak validation fold yang belum dipakai belajar pada putaran tersebut.

Lihat `KODE_ML/research/validation.py`, fungsi `cross_validate`, baris 34-41:

```
for fold_index, fold in enumerate(folds[name]):
    tr = np.array([positions[i] for i in fold["train"]], dtype=int)
    va = np.array([positions[i] for i in fold["validation"]], dtype=int)
    model = fit(algorithm, params, x[tr], y[tr, j], limits)
    probabilities = model.predict_proba(x[va])[:, 1]
    decisions = (probabilities >= 0.5).astype(np.int8)
    metrics = binary_metrics(y[va, j], probabilities, decisions)
    fold_values.append(metrics)
```

`tr` menyimpan posisi baris latihan; `va` posisi validasi. `x[tr]` berukuran `(jumlah_train_fold, jumlah_fitur)`; `y[tr, j]` hanya satu label. `predict_proba` menghasilkan dua skor per baris; `[:, 1]` mengambil skor positif. Batas 0,5 menghasilkan keputusan 0/1. Jumlah baris fold mengikuti pembagian kelompok sehingga tidak harus persis sama.

Pembagian dibuat **per label** dengan menjaga anggota kelompok molekul tetap bersama. Jadi jangan menganggap seluruh label pasti memakai satu daftar validation fold yang identik. Aturan ini menjaga anggota grup historis tetap bersama. Seluruh kemiripan struktur atau scaffold belum dijamin terpisah.

Seed dan OOF

`repeated_grouped_cv`, mulai baris 119, mengulang CV memakai seed **7, 21, 42, 84, 100**. Seed adalah angka pengendali pengacakan. Lima angka ini mengubah pembagian CV. Sementara itu `make_model` tetap memakai `random_state=42`. Jadi seed pembagian data dan seed classifier punya tugas berbeda.

OOF berarti prediksi untuk baris train dari model yang tidak melatih baris itu pada fold bersangkutan. Satu seed menghasilkan matriks OOF `(5353, 109)`. Lima hasil dirata-ratakan menjadi matriks berukuran sama. Kode memakainya untuk mencoba threshold per label yang memaksimalkan F1. Threshold ini berasal dari validasi, bukan test. Pelaporan utama dan inference bundle tetap memakai **0,5**.

Pemilihan memakai rata-rata macro AP kelima seed. Nilainya bisa berbeda dari AP atas rata-rata skor OOF: merata-ratakan metrik berbeda dengan merata-ratakan skor.

Model final dan batas kesimpulan

`KODE_ML/research/experiment.py`, `run_all`, baris 232-301 mengatur urutan ini. Setelah keputusan dibekukan, `fit_final_models` belajar ulang memakai seluruh train. Model final ini berbeda dari model sementara setiap fold. Test tidak ikut final fit.

Hasil tersimpan memilih **xgb_D** dalam XGBoost dan **lgbm_D** dalam LightGBM. Pilihan keseluruhan adalah **lgbm_D**. Bukti dibaca dari `complete.json`, melalui lokasi hasil yang ditetapkan `results.main_run()`, bukan menebak folder dengan angka versi terbesar.

Batasnya: repeated CV ini **bukan nested CV**. Data pengembangan turut dipakai mencari dan menilai resep sehingga hasil CV bersifat deskriptif. Protokol juga mencatat test historis pernah diperiksa; penelitian bersifat eksploratif. Jangan menyebutnya pembuktian konfirmatori tanpa batasan.

Bekal sidang

“Optuna mencari model atau label?” “Optuna mencari paket hyperparameter. Satu paket diuji pada 109 classifier label dan dinilai dengan macro AP validasi.”

“Kenapa final training setelah CV?” “CV membantu memilih resep tanpa melatih validation fold pada putarannya. Setelah resep dikunci, model final belajar dari seluruh train agar semua data latih dimanfaatkan.”

Cek pemahaman

- [ ] Aku bisa membedakan trial, fold, seed, dan kandidat.
- [ ] Aku tahu target Optuna adalah AP, bukan logloss.
- [ ] Aku tahu test tidak dipakai untuk mencari threshold dalam run ini.
- [ ] Aku bisa menjelaskan batas repeated CV yang tidak nested.

08 - Cara baca hasil: model kita bagusnya di bagian mana?

Target belajar: hitung metrik, baca hasil asli, jelaskan pemilihan model. Angka contoh di awal adalah **ilustrasi**, bukan hasil penelitian.

Mulai dari satu aroma dulu

Bayangkan kita mengecek label `sweet` pada enam molekul. Target **1** berarti label tercatat di dataset; **0** berarti tidak tercatat. Nol belum membuktikan molekul benar-benar tidak beraroma sweet.

Model menghasilkan **skor**, lalu aturan ambang mengubahnya jadi keputusan. Dengan ambang 0,5, skor minimal 0,5 menjadi prediksi 1.

Molekul ilustrasi	Target	Skor sweet	Prediksi	Nama hasil
A	1	0,9	1	TP
B	0	0,8	1	FP
C	1	0,7	1	TP
D	0	0,6	1	FP
E	1	0,4	0	FN
F	0	0,1	0	TN

TP: prediksi 1 dan target 1. **FP**: prediksi 1 tetapi target 0. **FN**: prediksi 0 tetapi target 1. **TN**: prediksi 0 dan target 0. Kita dapat TP=2, FP=2, FN=1, TN=1.

Hitung metriknya pelan-pelan

Huruf pada rumus adalah jumlah kejadian tadi.

Metrik	Rumus	Hitungan ilustrasi	Pertanyaan yang dijawab
Precision	TP/(TP+FP)	2/4 = **0,50**	Dari yang diprediksi sweet, berapa bagian yang cocok dengan dataset?

Metrik	Rumus	Hitungan ilustrasi	Pertanyaan yang dijawab
Recall	TP/(TP+FN)	2/3 = **0,667**	Dari semua sweet tercatat, berapa bagian yang tertangkap?
F1	2TP/(2TP+FP+FN)	4/7 = **0,571**	Bagaimana gabungan precision dan recall?
Specificity	TN/(TN+FP)	1/3 = **0,333**	Dari target 0, berapa bagian yang tetap diprediksi 0?
Accuracy	(TP+TN)/(TP+FP+FN+TN)	3/6 = **0,50**	Berapa bagian seluruh keputusan yang cocok?

F1 juga bisa dihitung sebagai **2PR/(P+R)**. P berarti precision dan R berarti recall untuk **label yang sama**. Jangan menghitung macro F1 dari macro precision dan macro recall: hitung F1 setiap label dulu, baru rata-ratakan.

Di kode, metrics.py baris 10 menerima tiga vektor: `truth` = target asli, `probabilities` = skor, `predicted` = keputusan. Untuk satu label test, masing-masing panjangnya 1.333. Implementasi precision dan recall memakai scikit-learn; kasus pembagi nol dikembalikan sebagai nol melalui `zero_division=0`.

Potongan asli validation.py baris 69:

```
def f1_from_metrics(value):
    precision, recall = value["precision"], value["recall"]
    return 2 * precision * recall / (precision + recall) if precision + recall else 0.0
```

`precision` adalah P dan `recall` adalah R. Kondisi terakhir mencegah pembagian nol.

AP dan AUROC melihat urutan skor

AP, Average Precision, mengecek apakah target positif muncul di peringkat atas. Pakai tabel ilustrasi yang sudah diurutkan dari skor terbesar. Ada tiga positif, di peringkat 1, 3, dan 5. Precision sampai posisi itu adalah 1/1, 2/3, dan 3/5.

AP = (1 + 2/3 + 3/5)/3 = 0,756.

Pembagi 3 adalah jumlah positif. Rumus sederhana ini berlaku untuk contoh tanpa skor kembar. Secara umum AP menjumlahkan precision yang dibobot kenaikan recall. Kode benar-benar memanggil `average_precision_score(y, p)` di metrics.py baris 23. Kolom `auprc` dalam artefak berarti **AP dari fungsi tersebut**, bukan integrasi trapesium kurva precision-recall.

AUROC bisa dibayangkan dengan memasangkan setiap positif melawan setiap negatif. Positif punya skor 0,9; 0,7; 0,4. Negatif punya 0,8; 0,6; 0,1. Ada 3×3=9 pasangan. Skor positif menang masing-masing 3, 2, dan 1 kali. Jadi **AUROC = 6/9 = 0,667**. Skor sama mendapat setengah kemenangan. Implementasinya `roc_auc_score(y, p)` di baris 26.

AP dan AUROC memakai skor, bukan keputusan ambang 0,5. Mengganti ambang saja tidak mengubah keduanya. Contoh tadi juga menunjukkan AP dan AUROC bukan angka yang sama.

Dari satu label ke 109 label

Kode menghitung metrik **per label**, lalu `summarize()` di metrics.py baris 33 mengambil rata-ratanya. Itulah **macro**. Secara umum helper mengabaikan nilai metrik None jika suatu kelas tidak tersedia. Pada test utama, seluruh 109 label mempunyai kedua kelas sehingga rata-rata memakai 109 label. Contoh: AP sweet 0,6 dan woody 0,2 menghasilkan macro AP (0,6+0,2)/2=0,4. Setiap label mendapat bobot sama, meskipun jumlah contohnya berbeda.

Untuk seleksi, urutannya: **AP setiap fold → rata-rata fold tiap label → rata-rata label → rata-rata lima seed**. Seed yang dipakai 7, 21, 42, 84, 100; fold maksimal lima per label. Lihat `cross_validate()` baris 8 dan `repeated_grouped_cv()` baris 119 pada validation.py.

Ini beda dengan menggabungkan seluruh skor **OOF** lalu menghitung AP sekali. OOF berarti skor suatu baris dibuat saat baris itu menjadi validasi. Kode merata-ratakan skor OOF antarseed untuk memilih ambang tambahan. Karena AP bergantung peringkat, **rata-rata AP fold tidak harus sama dengan AP gabungan OOF**.

Hasil asli: delapan kandidat utama

A/C memakai Morgan 2.048 fitur. B/D menambah delapan deskriptor, menjadi 2.056 fitur. A/B memakai baseline; C/D memakai parameter hasil Optuna. Tabel berasal dari comparison_all.csv. Angka dibulatkan enam desimal; F1 memakai ambang utama 0,5.

Kandidat	AP validasi	AP test	AUROC test	Macro F1 test
XGBoost A	0,249589	0,249043	0,809232	0,274899
XGBoost B	0,277412	0,288125	0,822636	0,311835
XGBoost C	0,267049	0,273029	0,825034	0,270369
XGBoost D	0,296959	0,308827	0,838546	0,305902
LightGBM A	0,251681	0,252837	0,809417	0,277184
LightGBM B	0,283211	0,296380	0,825813	**0,320721**
LightGBM C	0,269114	0,276461	0,827912	0,252220
LightGBM D	**0,299731**	**0,310531**	**0,842985**	0,293156

LightGBM D dipilih dari AP validasi tertinggi, sesuai experiment.py fungsi select_candidates(), baris 66. AP 0,310531 bukan berarti "akurasi aroma 31,05%". Itu ringkasan kualitas peringkat skor pada label test.

D nggak unggul di semua metrik. LightGBM B memiliki F1 lebih tinggi pada ambang 0,5. Peringkat yang membaik belum tentu memberi keputusan terbaik pada ambang tetap. Dibanding B, D menaikkan recall tetapi menurunkan precision.

Kenapa ada hasil ambang OOF?

`select_validation_thresholds()` di validation.py baris 89 mencoba ambang 0,05 sampai 0,95, langkah 0,05, lalu memilih F1 OOF tertinggi **masing-masing label**. Kalau seri, pilih yang paling dekat 0,5; kalau masih seri, pilih yang lebih besar. Ini evaluasi tambahan; hasil utama tetap ambang 0,5.

Hasil LightGBM D dari test_summary.json dan test_summary_validation_thresholds.json:

Metrik test	Ambang 0,5	Ambang dari OOF
Macro precision	0,213862	0,303265
Macro recall	0,515089	0,411890
Macro F1	0,293156	0,330419
Rata-rata label diprediksi/molekul	9,355589	6,372093

Model jadi lebih selektif secara keseluruhan: precision naik, tetapi lebih banyak positif terlewat. Target test rata-rata 4,254314 label per molekul. AP dan AUROC tetap karena skor tidak berubah. Nama

`true_positive_rate` pada bagian `diagnostics` JSON dihitung sebagai `np.mean(truth)`: itu **proporsi target positif**, bukan recall.

Batas kesimpulan dan latihan

Menurut experiment.json baris 171, test pernah diperiksa pada riwayat pengembangan dan repeated CV ini bukan nested CV. Jadi hasil bersifat **eksploratori**. Simpangan baku AP validasi D sebesar 0,001360 menggambarkan variasi lima seed; itu bukan bukti perbedaan signifikan atau jaminan pada molekul baru.

Audit 27 Oktober juga membandingkan prediksi semua nol: macro accuracy-nya **0,960970**, melampaui D **0,923660**, tetapi recall dan F1 nol. Ini alasan konkret kenapa accuracy bukan dasar seleksi.

Jawaban sidang singkat: "Saya memilih LightGBM D berdasarkan macro AP validasi. AP test-nya 0,310531. Pada ambang 0,5, F1-nya masih di bawah baseline B. Ambang tambahan dari OOF meningkatkan F1 dengan konsekuensi recall turun. Saya melaporkan kedua kondisi dan batas evaluasinya."

Cek paham: pakai ambang 0,75 pada tabel ilustrasi. Hasilnya TP=1, FP=1, FN=2, TN=2. Kenapa recall berubah tetapi AP tetap? Cari pembanding ambang di notebook 07.

9 Menjalankan project dan prediksi

Yang perlu kamu pahami

Targetnya: membuka panduan dan menjelaskan prediksi. **Fit** berarti belajar; **predict** memakai pola yang sudah dipelajari. Prediksi tidak melatih 109 model dari awal.

Buka dari tempat yang tepat

Buka folder utama repository:

```
C:\Users\ACER\Documents\Marvel\Skripsi\Project Skripsi\Perfume-MultiLabel-Classifier
```

Gunakan interpreter `.venv` project. Baca `KODE_ML/README.md`, lalu delapan notebook `01_...ipynb` sampai `08_Prediksi_Molekul.ipynb`. Setiap notebook dirancang mandiri dari **kernel baru**, yaitu sesi Python kosong. Audit 27 Oktober menemukan alat kernel notebook belum terpasang di environment ML lokal. Output tersimpan tetap bisa dibaca. Untuk menjalankan cell, ikuti pemasangan `KODE_ML/requirements_belajar.txt` pada README; panduan ini tidak memasang dependency.

Notebook memakai helper `belajar.tampilkan_sumber` untuk menampilkan fungsi sumber aktif, lokasi, nomor baris, dan sidik jari kodenya. Jadi ikuti sumber `KODE_ML/research`, bukan mengira seluruh cell merupakan salinan training mandiri. Notebook prediksi menjalankan inferensi ringan dari model yang sudah ada.

Kalau mau menjalankan sel, langkahnya dari PowerShell di root repository:

```
.\.venv\Scripts\python.exe -m pip install -r KODE_ML\requirements_belajar.txt
```

Perintah ini petunjuk pemasangan untuk kamu nanti, bukan dijalankan saat panduan dibuat. Di VS Code buka notebook01, pilih Select Kernel -> Python Environments -> interpreter `.venv\Scripts\python.exe`. Pilih Restart Kernel, lalu Run All. Tahap berhasil jika sel selesai tanpa traceback; notebook01 menampilkan tabel sumber, notebook03 ukuran fitur/target yang sesuai, dan notebook07 menampilkan delapan kandidat. Jika tidak ada `.venv` atau artefak model/data hilang, berhenti dan periksa kebutuhan project; jangan menganggap cukup memasang library notebook untuk membangun ulang seluruh environment penelitian.

Sumber aktif dirapikan setelah eksperimen kampus; bukti kesetaraan fungsi dan artefak ada di `HASIL_VERIFIKASI.md`. Ini bukan file literal yang tetap sama sejak training kampus. Hasil historis dikunci terpisah dari run baru.

Periksa input tanpa training

Command berikut ditulis untuk PowerShell dari root repository. Ini petunjuk; tidak dijalankan ketika menyusun panduan ini.

```
.\.venv\Scripts\python.exe -B -m research.experiment --check
```

Lihat `KODE_ML/research/experiment.py`, fungsi `main`, baris 302-313. Tanpa `--train`, CLI juga masuk jalur pemeriksaan. Output sukses mencantumkan `MAIN_M2048_D8_INPUTS_VERIFIED` dan `training_started: False`. Pemeriksaan ini memastikan input sesuai protokol; ini bukan evaluasi akurasi baru.

Di notebook `07_Evaluasi_dan_Pemilihan.ipynb`, gate `JALANKAN_EKSPERIMEN_BARU = False` harus tetap False untuk belajar. Training baru memerlukan tugas dan izin terpisah, folder run baru, serta lolos `runtime.check_host` baris 49-55. Guard memblokir laptop asal yang dicatat protokol. Profil yang didukung adalah CPU. Jangan mengubah guard demi menjalankan cell belajar.

Ikuti satu SMILES sampai jadi fitur

Sumber: `KODE_ML/research/inference.py`, fungsi `predict`. Potongan asli baris 31-39:

```
mol = Chem.MolFromSmiles(smiles)
if mol is None or len(Chem.GetMolFrags(mol)) != 1:
    raise ValueError("Input must be one valid connected molecule.")
generator = rdFingerprintGenerator.GetMorganGenerator(
    radius=spec["radius"], fpSize=spec["n_bits"], includeChirality=spec["include_chirality"])
x = np.concatenate([generator.GetFingerprintAsNumPy(mol),
                    [getattr(Descriptors,name)(mol) for name in
names]]).astype(np.float32)[None,:]
if not np.isfinite(x).all() or x.shape[1] != spec["dimensions"]["input"]:
    raise ValueError("Feature schema mismatch.")
```

RDKit mengubah teks menjadi objek molekul. Input harus satu molekul terhubung yang valid. Pengaturan Morgan dan urutan deskriptor dibaca dari bundle model. NumPy menggabungkan nilainya. `[None, :]` membuat bentuk satu baris: `(1, 2056)` untuk kandidat utama lgbm_D. Ini urutan fitur yang sama dengan saat training. SMILES valid dan terhubung belum membuktikan molekul sesuai domain bahan pewangi. Audit 27 Oktober mencatat batas input mode nama dan SMILES belum konsisten; struktur valid saja bukan jaminan prediksi dapat dipercaya.

Sebelumnya, baris 18-30 memeriksa bundle, versi library, dan spesifikasi fitur. Bila gagal, cocokkan environment dan artefaknya; jangan langsung mematikan pemeriksaan.

Bagaimana 109 skor keluar

Potongan asli baris 53-58:

```
        score = float(model.predict_proba(x)[0,1])
    output.append({"label":label, "score":score,
"predicted":score>=bundle["primary_threshold"]})
return {"smiles":Chem.MolToSmiles(mol), "candidate":bundle["selected_candidate"],
        "descriptors":len(names), "threshold":bundle["primary_threshold"],
        "score_meaning":"uncalibrated model score, not aroma intensity",
        "labels":sorted(output,key=lambda row:row["score"],reverse=True)}
```

Bagian ini berada dalam loop semua label. Sebelumnya `joblib.load` membuka model label dan memeriksa sidik jari serta jumlah fitur. `[0, 1]` mengambil skor kelas positif untuk baris molekul pertama. `predicted` bernilai True jika skor minimal 0,5. Daftar akhirnya diurutkan dari skor terbesar.

Contoh **ilustrasi**: sweet 0,82, woody 0,17, floral 0,61. Keputusannya sweet dan floral positif. Boleh menampilkan 0,82 sebagai "skor 82%", tetapi **bukan akurasi 82%**. Skor belum dikalibrasi: belum dibuktikan bahwa seratus prediksi berskor 82% akan tepat 82 kali. Skor juga bukan intensitas aroma. Jumlah skor seluruh label tidak harus 100% karena beberapa aroma bisa positif bersamaan.

Command prediksi dari model tersimpan

```
.\.venv\Scripts\python.exe -B -m research.inference --smiles "COc1cc(C=O)ccc1O"
```

Ini contoh SMILES vanillin. CLI `main`, baris 61-67, memakai `results.main_run()` bila `--run` tidak diberikan. Fungsi tersebut menunjuk hasil utama yang dikunci. Bila hasil tidak ditemukan, program berhenti; tidak otomatis training. Perintah ini menghasilkan JSON berisi SMILES, kandidat, threshold, dan 109 skor.

Vanillin tercatat dalam dataset, jadi contoh ini bukan bukti kemampuan pada molekul baru. Evaluasi resmi memakai test terpisah.

Hubungannya dengan aplikasi

Audit `reports/final_audit_20261027/README.md` membuktikan prototipe memakai API RDKit untuk fitur dan 109 ONNX di aplikasi. API aktif ialah `feature_api` di repo frontend, berbeda dari API lama repo ML. Uji desktop 109 model × 32 vektor menghasilkan galat maksimum `4,001013e-7`, dengan keputusan 0,5 sama. Demo Android fisik pernah dicatat, tetapi audit ini tidak menguji ulang HP. Jadi integrasi tersedia dan diuji terbatas; hosting produksi dan pengujian luas belum selesai.

Bekal sidang

“Apakah input baru membuat model belajar lagi?” “Tidak. SMILES diubah menjadi fitur dengan skema training, kemudian 109 classifier tersimpan menghitung skor. Tidak ada pemanggilan fit.”

“Kenapa versi RDKit diperiksa?” “Agar pembentukan fitur konsisten dengan training. Model mengharapkan jumlah dan urutan fitur tertentu, jadi perubahan pembentukan fitur bisa mengubah hasil.”

Cek pemahaman

- [ ] Aku tahu perbedaan check, fit, dan predict.
- [ ] Aku bisa menunjukkan perubahan teks SMILES menjadi `(1, 2056)`.
- [ ] Aku tahu lokasi pemilihan model utama dan threshold.
- [ ] Aku bisa membaca skor tanpa menyebutnya akurasi atau intensitas.

10 - Latihan jawab dosen pakai pemahaman sendiri

Status aplikasi terbaru ada di HASIL_UNTUK_BELAJAR/audit_27_oktober/README.md; catatan penundaan ONNX pada plan lama sudah usang.

Target belajar: jelaskan alur, tunjukkan bukti, sebutkan batasnya. Sesuaikan jawaban dengan pekerjaan yang benar-benar kamu lakukan.

Pegangan utama: SAFE_EXECUTION_PLAN.md, config penelitian, serta kode aktif di `KODE_ML/research/`. Nama arsip V7 menunjukkan hasil yang dikunci, bukan nama algoritma baru. V9 adalah eksperimen tambahan.

Putaran pertama: jelaskan alurnya

1. Penelitianmu memprediksi apa?

"Beberapa label aroma untuk satu struktur molekul. Satu molekul bisa punya beberapa label, jadi tugasnya multilabel." Ini prediksi dari anotasi, bukan pengukuran aroma atau prediksi campuran parfum.

2. Bentuk input dan outputnya apa?

"Input berupa SMILES. Struktur diubah menjadi fitur angka, kemudian model menghasilkan 109 skor label." Untuk LightGBM D, satu input berbentuk **1×2.056 fitur**, keluar **1×109 skor**. Jika ditampilkan persen, 0,7 menjadi 70%; itu tetap skor model yang belum dikalibrasi.

3. Model membaca huruf SMILES langsung?

"RDKit membaca struktur SMILES, lalu menghitung Morgan dan deskriptor." Bukti: features.py, calculate_morgan(), baris 29 dan `calculate()` baris 19. Struktur tidak valid atau multikomponen ditolak.

4. Kenapa 2.048+8?

"Morgan merangkum pola lingkungan atom. Deskriptor menambah sifat molekul, misalnya berat molekul dan logP. Manfaat tambahannya diuji lewat perbandingan skenario." Jangan bilang 2.048 pasti optimal dibanding semua ukuran.

5. Apa arti label 0 dan 1?

"Satu berarti aroma tercatat setelah pemetaan label. Nol berarti tidak tercatat." Jangan menjawab "nol pasti tidak berbau itu", karena anotasi sumber bisa belum lengkap dan label aroma bersifat subjektif.

6. Berarti ada 109 pohon?

"Ada 109 classifier biner untuk satu kandidat algoritma. Setiap classifier punya banyak pohon boosting." XGBoost dan LightGBM dibandingkan sebagai dua pilihan algoritma. Satu prediksi dengan kandidat terpilih menggunakan 109 classifier kandidat itu. Lihat training.py, fit_final_models(), baris 55.

7. Pohon-pohon itu bekerjanya gimana?

"Pohon membagi data berdasarkan fitur. Dalam boosting, pohon berikutnya menambah koreksi terhadap prediksi gabungan sebelumnya berdasarkan loss." Pembelajarannya menggunakan turunan loss, bukan sekadar menghitung tebakan salah.

8. Kalau labelnya jarang muncul gimana?

"Kode memberi bobot sampel berdasarkan frekuensi kelas pada bagian training yang sedang dipakai. Kelas langka mendapat bobot lebih besar." Rumusnya **bobot kelas = N/(2×jumlah kelas itu)**; N adalah jumlah sampel training. Jika N=100 dan positif=10, bobot positif=5, negatif=100/180≈0,556. Bukti: training.py, balanced_sample_weights(), baris 21. Ini bobot saat menghitung loss, bukan membuat molekul baru.

Putaran kedua: alasan eksperimen dan hasil

9. A, B, C, D bedanya apa?

Skenario	Fitur	Parameter
A	Morgan 2.048	Baseline
B	Morgan 2.048 + 8 deskriptor	Baseline
C	Morgan 2.048	Hasil Optuna
D	Morgan 2.048 + 8 deskriptor	Hasil Optuna

“Ada delapan kandidat. A lawan B memeriksa tambahan deskriptor; B lawan D memeriksa tuning dengan fitur sama.” Aturan bobot kelasnya sama.

10. Optuna memilih berdasarkan apa?

“Setiap trial mencoba satu kombinasi parameter. Nilainya dihitung dari rata-rata AP fold per label, lalu rata-rata seluruh 109 label pada data training. Nilai lebih tinggi lebih baik.” Empat studi C/D mendapat anggaran sama, delapan jam per studi. Bukan memilih parameter dari AP test. Detailnya di tuning.py.

11. Train, validasi, test bedanya apa?

“Training untuk mempelajari pola. Validasi untuk menilai pilihan saat pengembangan. Test untuk pelaporan setelah pilihan dibekukan.” Data utama berisi 6.686 molekul: 5.353 train dan 1.333 test. Jangan menyebut semua data dipakai melatih model final penelitian.

12. Kenapa pakai grup? Ada jejak 1.024 bit?

“Pembagian mempertahankan grup historis supaya anggota grup tidak menyeberang train dan test. Asal grup memakai kesamaan struktur tanpa stereo atau fingerprint Morgan 1.024 yang identik. Fitur model utama sekarang tetap Morgan 2.048.” Jangan bilang seluruh molekul mirip pasti terpisah; aturan ini hanya melindungi grup yang didefinisikan. splits.py, validate_grouped_split(), baris 21 memeriksa pemisahannya.

13. Seed itu apa, dan apakah semua label punya fold sama?

“Seed mengendalikan pengacakan. Evaluasi memakai lima seed; fold dibuat per label memakai target dan grup, jadi antarlabel bisa berbeda.” Kode mengizinkan pengurangan fold saat kelas tidak mencukupi; audit hasil utama menemukan lima fold setiap label/seed. Lihat splits.py, inner_folds(), baris 6.

14. Kenapa LightGBM D dipilih?

“Karena mean macro AP validasinya paling tinggi, 0,299731. AP test-nya 0,310531 dan AUROC test-nya 0,842985.” Tambahkan: D tidak tertinggi pada semua metrik; F1 ambang 0,5 LightGBM B lebih tinggi. Jangan mengganti aturan seleksi sesudah membaca test.

15. Kenapa accuracy tinggi tetapi precision rendah?

“Sebagian besar pasangan molekul-label bernilai nol. Banyak keputusan negatif yang benar bisa membuat accuracy tinggi. Precision menyorot prediksi positif, jadi mengungkap masalah yang bisa tertutup oleh accuracy.” Untuk D pada 0,5, macro accuracy 0,923660 tetapi precision 0,213862. Karena itu angka accuracy saja tidak cukup.

16. Ambang OOF bikin semua metrik naik?

“Tidak. Pada LightGBM D, macro F1 naik dari 0,293156 menjadi 0,330419, tetapi recall turun dari 0,515089 menjadi 0,411890. Ambangnya dipilih dari validasi OOF training, bukan test.” AP dan AUROC tetap karena urutan skor tidak diubah.

Putaran ketiga: batas dan tanggung jawab

17. Berarti model terbukti lebih baik dari penelitian sebelumnya?

“Saya membatasi kesimpulan pada kandidat dan dataset penelitian ini. Perbedaan sumber data, label, pembagian, dan protokol membuat angka lintas penelitian belum tentu sebanding.” Test pernah diperiksa dalam riwayat pengembangan; repeated CV bukan nested CV. Selisih kecil dan simpangan baku lima seed belum membuktikan signifikansi statistik.

18. Apakah fitur yang sama selalu punya label sama?

“Tidak. Audit menemukan 616 kelompok fitur identik, meliputi 1.474 molekul, dengan target berbeda. Model deterministik tidak bisa menghasilkan dua profil berbeda dari fitur yang sama.” Sebanyak 605 kelompok konfliknya berstruktur sama setelah stereo dihapus; 11 masih berbeda. Tidak ada kelompok fitur identik melintasi train-test. Ini batas representasi/anotasi, bukan bukti semua molekul tersebut salah diprediksi.

19. Kalau kode dibantu AI, kontribusimu apa?

“Saya menggunakan AI untuk membantu penulisan dan penjelasan kode. Saya bertanggung jawab memahami rancangan, memeriksa implementasi, dan menilai hasilnya.” Sebutkan **pekerjaan yang benar-benar sudah dilakukan**, misalnya menelusuri fungsi atau menghitung metrik. Kalau belum dilakukan, jangan diklaim. Hafalan tidak menggantikan pemeriksaan kode.

20. Aplikasinya sudah terbukti siap produksi?

“Integrasi prototipe tersedia. Audit desktop mengecek 109 ONNX pada 32 vektor: selisih skor maksimum $4{,}001013 \times 10^{-7}$ dan keputusan ambang 0,5 sama dengan native.” Bukti Android sebelumnya tersedia, tetapi audit ini tidak menguji ulang HP. Pengujian terbatas belum membuktikan kesiapan produksi atau validitas sensoris. Rujukan temuan dan batas terbaru: audit 27 Oktober.

Kamus cepat

Istilah	Arti gampangnya
SMILES	Teks untuk menuliskan struktur molekul
Fitur	Angka yang dibaca model sebagai masukan
Morgan	Representasi bit dari pola lingkungan atom
Deskriptor	Angka sifat molekul yang dihitung RDKit
Label	Nama aroma yang menjadi target
Multilabel	Satu data boleh memiliki beberapa label
Classifier biner	Model untuk satu target 0/1
Boosting	Menambahkan pohon koreksi secara bertahap
Loss	Ukuran kesalahan yang diminimalkan saat belajar
Parameter model	Nilai yang dipelajari, misalnya nilai daun pohon
Hyperparameter	Pengaturan proses belajar, misalnya jumlah pohon
Trial	Satu percobaan kombinasi hyperparameter
Fold	Satu bagian pembagian validasi silang
Seed	Angka pengendali pengacakan
OOF	Prediksi baris ketika baris itu menjadi validasi
Threshold/ambang	Batas skor untuk memutuskan 0/1
Macro	Rata-rata metrik per label dengan bobot sama
Kalibrasi	Kesesuaian skor probabilitas dengan frekuensi kejadian nyata
Leakage	Informasi evaluasi bocor ke proses belajar/pemilihan
Overfitting	Pola training dipelajari terlalu khusus sehingga sulit berlaku pada data baru

Checklist: siap karena bisa melakukan

- [ ] Gambar alur SMILES → fitur → 109 classifier → skor → ambang → label.
- [ ] Tunjukkan fungsi asli pembuat fitur, pembobotan, pelatihan, dan metrik.
- [ ] Bedakan 109 classifier dari jumlah pohon masing-masing classifier.
- [ ] Hitung precision, recall, F1, dan AP contoh kecil tanpa bantuan kode.
- [ ] Temukan angka D dan B di CSV asli, lalu jelaskan kelebihan dan kekurangannya.
- [ ] Jelaskan kenapa skor 70% bukan accuracy 70% atau jaminan aroma ada.
- [ ] Sebutkan batas label nol, grup historis, dan test yang pernah dilihat.
- [ ] Akui bagian yang belum dipahami; jangan mengarang langkah yang belum dikerjakan.

Latihan: rekam jawaban nomor 2, 6, 14, dan 17 masing-masing 30-60 detik tanpa membaca. Cek ke kode dan artefak: apakah penjelasanmu punya bukti?

11. Peta source dan cuplikan aktual

Ini penunjuk file dan fungsi aktif pada 27 Oktober 2026. Semua lokasi di tabel dihitung langsung dari AST source. Cuplikan di bawah dibaca dari file tersebut, bukan ditulis ulang sebagai algoritma lain. Nomor baris dan SHA256 membantu mencocokkan versi; keduanya bukan parameter ML.

Source di research	Fungsi	Baris	Peran singkat
source_identity.py	normalize	32-36	Identitas dan istilah
source_identity.py	_attach_smiles	57-71	Identitas dan istilah
source_identity.py	_attach_sigma	78-92	Identitas dan istilah
preprocessing.py	load_sources	42-57	Baca dan bersihkan lima sumber
preprocessing.py	map_labels	67-79	Baca dan bersihkan lima sumber
preprocessing.py	tokens_for	82-94	Baca dan bersihkan lima sumber
preprocessing.py	merge_sources	97-164	Baca dan bersihkan lima sumber
preprocessing.py	review_frozen_preprocessing	166-176	Baca dan bersihkan lima sumber
features.py	calculate	19-26	Struktur menjadi angka
features.py	calculate_morgan	29-33	Struktur menjadi angka
features.py	features	41-45	Struktur menjadi angka
dataset.py	verify	14-79	Kontrak data beku
dataset.py	load_partition	82-93	Kontrak data beku
dataset.py	encode_frozen_records	96-109	Kontrak data beku
splits.py	inner_folds	6-18	Grup dan fold
splits.py	validate_grouped_split	21-52	Grup dan fold
training.py	make_model	9-18	Classifier dan fitting
training.py	balanced_sample_weights	21-30	Classifier dan fitting
training.py	fit	33-53	Classifier dan fitting
training.py	fit_final_models	55-66	Classifier dan fitting
tuning.py	atomic_dump	11-15	Pencarian pengaturan
tuning.py	event	17-23	Pencarian pengaturan
tuning.py	suggest	25-36	Pencarian pengaturan
tuning.py	initialize	38-59	Pencarian pengaturan
tuning.py	tune	72-134	Pencarian pengaturan
validation.py	cross_validate	8-66	CV dan OOF
validation.py	f1_from_metrics	69-71	CV dan OOF

Source di research	Fungsi	Baris	Peran singkat
validation.py	prediction_diagnostics	74-86	CV dan OOF
validation.py	select_validation_thresholds	89-116	CV dan OOF
validation.py	repeated_grouped_cv	119-180	CV dan OOF
experiment.py	verify_files	32-34	Urutan eksperimen
experiment.py	preflight	36-63	Urutan eksperimen
experiment.py	select_candidates	66-71	Urutan eksperimen
experiment.py	evaluate_test	73-109	Urutan eksperimen
experiment.py	paired_cv	111-137	Urutan eksperimen
experiment.py	summaries	140-219	Urutan eksperimen
experiment.py	reject_historical_run	222-229	Urutan eksperimen
experiment.py	run_all	232-300	Urutan eksperimen
experiment.py	main	302-313	Urutan eksperimen
metrics.py	binary_metrics	10-30	Evaluasi
metrics.py	summarize	33-44	Evaluasi
metrics.py	compare	47-54	Evaluasi
inference.py	predict	16-58	Prediksi tersimpan
inference.py	main	61-67	Prediksi tersimpan
runtime.py	digest	19-20	I/O, hash dan penjagaan eksekusi
runtime.py	read	22-23	I/O, hash dan penjagaan eksekusi
runtime.py	write	26-31	I/O, hash dan penjagaan eksekusi
runtime.py	environment	34-36	I/O, hash dan penjagaan eksekusi
runtime.py	compute_profile	39-46	I/O, hash dan penjagaan eksekusi
runtime.py	check_host	49-54	I/O, hash dan penjagaan eksekusi
runtime.py	run_lock	58-72	I/O, hash dan penjagaan eksekusi
runtime.py	save_npz	93-99	I/O, hash dan penjagaan eksekusi
results.py	main_run	5-13	Alamat hasil utama

Cuplikan inti yang bisa kamu ikuti

Baca ini setelah bab 01-10. Source lengkap tetap ada di research dan ditampilkan notebook lewat tampilkan_sumber. Penjelasan input, output dan alasannya ada di bab terkait.

features.py: calculate_morgan

Baris 29-33. SHA256 file: `8a1774f10afe12c65985b1e326903f5509ad68814a61f6cb2d7df086f14564d0`.

```
def calculate_morgan(smiles):
    generator = rdFingerprintGenerator.GetMorganGenerator(
        radius=RADIUS, fpSize=N_BITS, includeChirality=False)
    return np.asarray([generator.GetFingerprintAsNumPy(molecule(smi, i))
                       for i, smi in enumerate(smiles)], dtype=np.uint8)
```

features.py: features

Baris 41-45. SHA256 file: `8a1774f10afe12c65985b1e326903f5509ad68814a61f6cb2d7df086f14564d0`.

```
def features(key, morgan, descriptors):
    if (morgan.ndim != 2 or morgan.shape[1] != N_BITS or descriptors.shape != (len(morgan), 8)
            or not np.isfinite(descriptors).all()):
        raise ValueError("Expected Morgan 2048 and eight ordered descriptors per molecular row.")
    return np.column_stack([morgan, descriptors[:, :descriptor_count(key)]]).astype(np.float32)
```

training.py: balanced_sample_weights

Baris 21-30. SHA256 file: `4b8f34ca70a8263b3a8a25e7c0e2b3e46bc1e8767be2149ec7cf4c804024e184`.

```
def balanced_sample_weights(y):
    """Use the same fold-local weighting rule for XGBoost and LightGBM."""
    values = np.asarray(y, dtype=np.int8)
    if values.ndim != 1 or len(values) == 0 or not np.isin(values, [0, 1]).all():
        raise ValueError("Expected a non-empty binary target vector.")
    counts = np.bincount(values, minlength=2)
    if np.any(counts == 0):
        raise ValueError("Balanced weighting requires both classes.")
    per_class = len(values) / (2.0 * counts)
    return per_class[values]
```

training.py: make_model

Baris 9-18. SHA256 file: `4b8f34ca70a8263b3a8a25e7c0e2b3e46bc1e8767be2149ec7cf4c804024e184`.

```
def make_model(algorithm, parameters):
    params = dict(parameters)
    profile = compute_profile()
    threads = profile["threads_per_model"]
    if algorithm == "xgb":
        return XGBClassifier(n_jobs=threads, random_state=42, eval_metric="logloss",
                             tree_method=profile["xgboost_tree_method"], **params)
    if algorithm == "lgbm":
        return LGBMClassifier(n_jobs=threads, random_state=42, verbosity=-1, **params)
    raise ValueError("Unknown algorithm.")
```

inference.py: predict

Baris 16-58. SHA256 file: `a9cdd7d49ed7a308134c9d6b82b47f1cbfcd720a9c58d02511ecf78a9b432a4a`.

```
def predict(run, smiles):
    run = Path(run)
    bundle = core.read(run/"inference_bundle.json")
    completion = core.read(run/"completion_manifest.json")["files"]
    if digest(run/"inference_bundle.json") != completion["inference_bundle.json"]:
        raise ValueError("Inference bundle changed.")
    if bundle["environment"] != core.environment():
        raise ValueError("Use the pinned training environment.")
    spec = bundle["feature_spec"]
    names = spec["descriptors"]
    if (names not in ([], list(DESCRIPTORS8)) or spec["n_bits"] != N_BITS
            or bool(names) != bundle["use_descriptors"]):
        raise ValueError("Unsupported descriptor order.")
    if spec.get("rdkit_version", rdBase.rdkitVersion) != rdBase.rdkitVersion:
        raise ValueError("RDKit version differs.")
    mol = Chem.MolFromSmiles(smiles)
    if mol is None or len(Chem.GetMolFrags(mol)) != 1:
        raise ValueError("Input must be one valid connected molecule.")
    generator = rdFingerprintGenerator.GetMorganGenerator(
```

```
    radius=spec["radius"], fpSize=spec["n_bits"], includeChirality=spec["include_chirality"])
x = np.concatenate([generator.GetFingerprintAsNumPy(mol),
                    [getattr(Descriptors,name)(mol) for name in
names]]).astype(np.float32)[None,:]
if not np.isfinite(x).all() or x.shape[1] != spec["dimensions"]["input"]:
    raise ValueError("Feature schema mismatch.")
output = []
for label in bundle["labels"]:
    entry = bundle["models"][label]
    path = run/bundle["models_directory"]/entry["file"]
    if digest(path) != entry["sha256"]:
        raise ValueError("Model checksum changed.")
    model = joblib.load(path)
    if model.n_features_in_ != x.shape[1]:
        raise ValueError("Model expects a different feature count.")
    # The ndarray feature order is frozen and checked above.
    import warnings
    with warnings.catch_warnings():
        warnings.filterwarnings("ignore", message="X does not have valid feature names, but
LGBMClassifier was fitted with feature names", category=UserWarning)
        score = float(model.predict_proba(x)[0,1])
    output.append({"label":label, "score":score,
"predicted":score>=bundle["primary_threshold"]})
return {"smiles":Chem.MolToSmiles(mol), "candidate":bundle["selected_candidate"],
        "descriptors":len(names), "threshold":bundle["primary_threshold"],
        "score_meaning":"uncalibrated model score, not aroma intensity",
        "labels":sorted(output,key=lambda row:row["score"],reverse=True)}
```

12 Library dan cara membaca kode

Bedakan tiga hal dulu

Library adalah kumpulan alat siap pakai. **Import** membuka alat itu supaya bisa dipanggil kode. **Dependency** adalah paket yang disediakan environment; keberadaannya belum berarti seluruh jalur program memakai paket tersebut.

Daftar versi di bawah berasal dari `requirements_research.txt` dan `KODE_ML/requirements_belajar.txt`. Status terpasang diperiksa dari metadata `.venv` pada 27 Oktober 2026, tanpa mengimpor model atau menjalankan training. **Versi yang diminta file requirements tidak selalu sama dengan kondisi environment.**

Library penelitian

Seluruh paket pada tabel ini terpasang dengan versi yang tercantum. Kolom terakhir membedakan yang diimpor langsung oleh kode aktif dari paket lain di environment.

Paket dan versi	Nama saat import	Peran yang ditemukan dalam kode
NumPy 1.26.4	`numpy as np`	Mengolah matriks fitur, target, indeks, skor, serta file NPZ
pandas 2.2.3	`pandas as pd`	Membaca CSV, membersihkan tabel, dan menulis rangkuman hasil
scikit-learn 1.6.1	`sklearn`	Metrik dan `StratifiedGroupKFold`; helper asal kohor juga memakai `GroupShuffleSplit`
RDKit 2026.3.4	`rdkit`	Membaca struktur, kanonisasi, Morgan, dan delapan deskriptor
XGBoost 2.1.4	`xgboost`, `XGBClassifier`	Classifier XGBoost dan callback pemeriksaan batas komputasi

Paket dan versi	Nama saat import	Peran yang ditemukan dalam kode
LightGBM 4.6.0	`LGBMClassifier`	Classifier LightGBM
Optuna 4.5.0	`optuna`	Study, trial, pencarian hyperparameter, dan pembacaan hasil study
joblib 1.6.0	`joblib`	Menyimpan/membaca model dan keadaan sampler
psutil 7.0.0	`psutil`	Memeriksa proses, RAM, dan informasi CPU
SciPy 1.15.3	Tidak diimpor langsung	Tersedia sebagai dukungan numerik/dependency; jangan mengaku memanggil fungsi SciPy pada sumber aktif
iterative-stratification 0.1.9	Tidak diimpor langsung	Terdaftar di environment; split aktif memakai scikit-learn, bukan paket ini
PyYAML 6.0.2	Tidak diimpor langsung	Terdaftar; konfigurasi jalur aktif dibaca sebagai JSON
requests 2.32.5	Tidak diimpor langsung	Terdaftar; pembacaan snapshot lokal aktif bukan pengunduhan HTTP lewat requests

"Tidak diimpor langsung" berlaku untuk sumber penelitian aktif dan helper delapan notebook yang diperiksa. Ini bukan klaim bahwa paket tersebut tidak pernah dipakai skrip arsip atau dependency lain.

Alat notebook dan grafik

Alat	Versi yang diminta	Kondisi `.venv` saat diperiksa dan kegunaannya
ipykernel	7.4.0	Belum terpasang; menghubungkan sesi Python dengan notebook
nbformat	5.11.1	Terpasang 5.10.4; membaca/menulis struktur notebook
nbclient	0.11.0	Belum terpasang; menjalankan sel notebook secara terprogram
nbconvert	7.17.2	Belum terpasang; mengubah notebook menjadi format lain
IPython	Tidak dipatok langsung dalam dua requirements tersebut	Belum terpasang; `IPython.display` menampilkan tabel dan Markdown
Matplotlib	Tidak dipatok dalam dua requirements tersebut	Terpasang 3.10.6; `matplotlib.pyplot` dipakai cell grafik

Output notebook yang sudah tersimpan tetap bisa dibaca. Untuk menjalankan sel, ikuti petunjuk environment di README. Alat notebook tidak mengganti algoritma ML. `pytest` 8.4.2 juga terpasang dan diimpor oleh file tes; itu alat pemeriksaan, bukan pembuat prediksi aroma.

Yang tidak perlu di-install lewat pip

Python sudah membawa **standard library**. Pada sumber aktif, kelompoknya antara lain:

- `pathlib`, `os`, `shutil`: alamat file dan operasi berkas.
- `json`, `re`, `collections`: membaca JSON, aturan teks, serta penghitung/pengelompokan.

- `argparse`, `platform`, `datetime`, `time`, `contextlib`: argumen command, identitas komputer, waktu, dan pengelolaan konteks.
- `hashlib`, `importlib.metadata`, `warnings`: sidik jari file, informasi versi paket, dan pesan peringatan.
- `ast`, `inspect`, `sys`: dipakai helper/notebook untuk membaca struktur atau lokasi kode dan mengatur pencarian modul.

`research`, `belajar`, `audit_labels`, dan `asal_kohor` adalah **modul project sendiri**, bukan paket yang perlu dicari di pip. Titik pada `from .runtime import ROOT` berarti mengambil modul tetangga dalam paket yang sama.

Contoh membaca fungsi per baris

Buka `KODE_ML/research/features.py`, fungsi `molecule`, baris 12-16. Ini potongan asli:

```
def molecule(smiles, index):
    mol = Chem.MolFromSmiles(smiles)
    if mol is None or len(Chem.GetMolFrags(mol)) != 1:
        raise ValueError(f"Invalid or multicomponent structure at row {index}; no rows may be
    dropped.")
    return mol
```

1. **Baris 12:** membuat fungsi milik project. Inputnya teks `smiles` dan nomor baris `index`.
2. **Baris 13:** memanggil RDKit melalui `Chem`. Hasilnya disimpan sebagai objek molekul `mol`, bukan fingerprint.
3. **Baris 14:** mengecek apakah pembacaan gagal atau jumlah komponen bukan satu. `or` berarti salah satu kondisi cukup untuk ditolak.
4. **Baris 15:** menghentikan proses dengan pesan berisi nomor baris. Data bermasalah tidak diam-diam dibuang.
5. **Baris 16:** jika valid, mengembalikan objek molekul untuk dihitung fitur berikutnya.

Jawaban sidang: "`molecule` fungsi project saya. Fungsi ini memakai RDKit untuk membaca SMILES dan memastikan satu struktur terhubung. Outputnya objek molekul, lalu fungsi fitur menghitung Morgan dan deskriptor."

Cek paham: tunjukkan mana nama library, alias, fungsi project, input, pemanggilan library, kondisi penolakan, dan outputnya.